COMSATS University Islamabad, Lahore Campus

Department of Computer Engineering

CPE 434 – VLSI DESIGN

LAB MANUAL FOR SPRING 2026

Lab Resource Person

Engr. Zaid Bin Javaid

Theory Resource Person

Dr. Muhammad Naeem Awais

Supervised By

Dr. Jehangir Arshad

Name: _____ **Registration Number:** *CUI/ - - /LHR*

Program: _____ **Batch:** _____

Semester _____

Revision History

S. No.	Activity	Date (DD/MM/YYYY)	Performed by
1	Lab Manual Preparation	06/03/2015	Engr. Irfan Ali Engr. Ehsanullah
2	Lab Manual review	14/03/2015	Dr. M. Naeem Awais Dr. Ahmad Saeed
3	Lab Manual review	10/02/2022	Engr. Muhammad Usman
4	Lab Manual review		

Preface

The past decade has witnessed the advent of VLSI designs. VLSI ASICs require a combination of high-speed performance, state-of-the-art packaging technology, and the integration of complex capabilities onto a single chip, all supported by design and development tools.

This Lab intend to give a broad overview of,

- The state of the art in VLSI design: layouts, circuits, logic, floor planning, and architectures
- New techniques for maximizing performance and minimizing power usage
- Extensive new coverage of advanced interconnect models, including copper

We have tried to present a comprehensive, "bottom-up" guide to the entire VLSI design process, focusing on the latest solutions for System-on-Chip (SoC) design. We have tried to review every aspect of digital design, from planning and layout to fabrication and packaging, introducing today's most advanced techniques for maximizing performance, minimizing power utilization, and achieving rapid design turnarounds. Coverage includes:

- Advanced interconnect models: new techniques for overcoming delay bottlenecks, reducing crosstalk, and modelling copper interconnect
- Advanced low-power design techniques for enhancing reliability and extending battery life in portable consumer electronics
- Testing solutions for every level of abstraction, from circuits to architecture
- Techniques for improving testability, embedded processors, and more

Books

Text Book

- Neil H.E. Weste and David Harris, "CMOS VLSI Design, A Circuits and Systems Perspective", 4th Edition, Addison Wesley, 2010
- Jan. M. Rabaey, A. Chandrakasan, and B.Nikolic, "Digital Integrated Circuits, A Design Perspective", 2nd Edition, Prentice Hall

Reference Books

- R. Jacob Baker, Harry W. Li, David E. Boyce, "CMOS Circuit Design, Layout, and Simulation", 3rdEdition, Wiley-IEEE Press, 2010
- N. H. E. Weste and K. Eshraghian, "Principles of CMOS VLSI Design: a Systems Perspective", 2ndEdition, by N. H. E. Weste and K. Eshraghian, Addison Wesley, 1992.

Learning Outcomes

Theory CLOs:

1. Comprehend the CMOS theory, quality matrix of CMOS circuit and to relate these concepts for the chip manufacturing process of integrated circuits using modern photolithographic fabrication process (C3-PLO1)

2. Analyze the design constraints including resources, delay & power consumption of combinational logic circuits, sequential logic circuits and large arithmetic blocks (C4-PLO2)

3. Design combinational logic circuits, sequential logic circuits and large arithmetic blocks with minimum resources, small delay & less power consumption using VLSI design circuit families (C5-PLO3)

Lab CLOs:

4. Design combinational and sequential logic circuits using EDA tools, considering minimal resource usage, low delay, and power efficiency. (C5-PLO3)

5. Utilize Electronic Design Automation (EDA) tools to analyze, verify, and reproduce digital circuit designs according to industry standards. (P3-PLO5)

6. Present the assigned project findings through a well-structured technical report, demonstrating clarity, coherence, and professionalism in communication. (A3-PLO10)

CLOs – PLOs Mapping

PLO / CLO	PLO1	PLO2	PLO3	PLO5	PLO10	Cognitive Domain	Affective Domain	Psychomotor Domain
CLO1	X					C3		
CLO2		X				C4		
CLO3			X			C5		
CLO4			X			C5		
CLO5				X				P3
CLO6					X		A3	

Lab CLOs – Lab Experiment Mapping

Lab / CLO	Lab 1	Lab 2	Lab 3	Lab 4	Lab 5	Lab 6	Lab 7	Lab 8	Lab 9	Lab 10	Lab 11	Lab 12
CLO4	P3	P3	P2	P2	P3	P3	P3	P3	P3	P3	P3	P3

Grading Policy

Lab Assignments: i. Lab Assignment 1 Marks (Lab marks from Labs 1-3) ii. Lab Assignment 2 Marks (Lab marks from Labs 4-6) iii. Lab Assignment 3 Marks (Lab marks from Labs 7-9) iv. Lab Assignment 4 Marks (Lab marks from Labs 10-12)	25%
Lab Mid Term = 0.5*(Lab Mid Term exam) + 0.5*(average of lab evaluation of Lab 1-6)	25%
Lab Terminal = 0.4*(Complex Engineering Problem) +0.1*(Open Ended Lab)+0.375*(average of lab evaluation of Lab 7-12) + 0.125*(average of lab evaluation of Lab 1-6)	50%
Total (lab)	**100%**

Complex Engineering Problem Lab [CEP]

List of equipment

Altera Cyclone IV E -DE2-115 FPGA, Altera Cyclone SoC V SE -DE1 FPGA

Software Resources

- DSCH
- Microwind
- Quartus 13.1

Lab Instructions

- This lab activity comprises of three parts: Pre-lab and Lab Tasks.
- Viv will be conducted during the lab performance.
- The students should perform and demonstrate each lab task separately for step-wise evaluation (please ensure that course instructor/lab engineer has signed each step after ascertaining its functional verification)
- Only those tasks that completed during the allocated lab time will be credited to the students. Students are however encouraged to practice on their own in spare time for enhancing their skills.

Lab Report Instructions

You must attach the results with each experiment. Lab report must be attached with the respective experiment with following items covered in it:

- Results
- Conclusion

Safety Instructions

- It is the duty of all concerned who use any electrical laboratory to take all reasonable steps to safeguard the health and safety of themselves and all other users and visitors.
- Be sure that all equipment is properly working before using them for laboratory exercises. Any defective equipment must be reported immediately to the Lab. Instructors or Lab. Technical Staff.
- Students are allowed to use only the equipment provided in the experiment manual.
- Power supply terminals connected to any circuit are only energized with the presence of the Instructor or Lab. Staff.
- Avoid any part of your body to be connected to the energized circuit and ground.
- Switch off the equipment and disconnect the power supplies from the circuit before leaving the laboratory.
- Observe cleanliness and proper laboratory housekeeping of the equipment and other related accessories.
- Make sure that the last connection to be made in your circuit is the power supply and first thing to be disconnected is also the power supply.
- Equipment should not be removed, transferred to any location without permission from the laboratory staffs.
- Students are not allowed to use any equipment without proper orientation and actual hands on equipment operation.
- Do not eat food, drink beverages, or chew gum in the laboratory.
- Report any accident or injury (cut, burn, etc.) to the teacher immediately, no matter how trivial it seems. Do not panic.

Table of Contents

Revision History 0

Preface **1**

Books **2**

Learning Outcomes **2**

CLOs – PLOs Mapping **3**

Lab CLOs – Lab Experiment Mapping **3**

Grading Policy **3**

Software Resources **4**

Lab Instructions **4**

Lab Report Instructions **4**

Safety Instructions **5**

LAB #1: **9**

Introduction to DSCH and Microwind simulation tools and *follow* the steps to implement the design of CMOS inverter **9**

Objective 9

Pre-Lab 9

In-Lab 10

Lab Tasks 18

LAB # 2: **20**

Part I: Introduction to VHDL and Altera Quartus tool and *follow* the steps to design static CMOS basic gates 20

Part II: To implement the design of static CMOS basic gates on Microwind and Altera Quartus tools and *reproduce* its results **20**

Objective 20

Pre-Lab 20

Part I: Layout design of CMOS Gates: 20

Part II: Familiarize yourself with VHDL and Quartus: 21

A. Steps to Create a New Project: 27

B. Create, Add, and Compile Design Files 29

C. Simulate Design using Quartus II 30

Lab Tasks 33

LAB # 3: 35

To *explain* the layout of static CMOS basic gates using bubble pushing and implement it on Microwind simulator 35

Objective 35

Pre-Lab 35

Lab Tasks 36

LAB # 4: 38

To *explain* the layout of 2 x 1 multiplexer using Transmission gate and Tristate buffer to implement its schematic on Microwind simulator and VHDL code on Quartus simulation tool to generate its vector waveform . 38

Objectives 38

Pre-Lab 38

Lab Tasks 41

LAB # 5: 43

To analyze λ - based design rules of CMOS transistor in Microwind, design stick diagrams of basic static CMOS gates and reproduce the results using Microwind simulator 43

Objective 43

Pre-Lab 43

Lab Tasks 47

LAB # 6: 50

To analyze the delay of CMOS gates in Microwind and reproduce the results using Microwind simulator 50

Objective 50

Pre-Lab 50

Lab Tasks 53

LAB # 7: 56

To implement the layout of combinational dynamic circuits in Microwind and *reproduce* its results 56

Objective 56

Pre-Lab 56

Lab Tasks 57

LAB # 8: 60

To implement the layout of combinational ratioed logic and *reproduce* the results using Microwind simulator 60

Objective 60

Pre-Lab 60
Lab Tasks 62
LAB # 9: 64
To implement the layout of D-latch, D- Flip flop and *reproduce* the results using Microwind simulator 64
Objective 64
Pre-Lab 64
Lab Tasks 66
LAB # 10: 70
To implement the layout of counters and *reproduce* the results using Microwind simulator 70
Objective 70
Pre-Lab 70
Lab Tasks 72
LAB # 11: 74
To implement the layout of registers and *reproduce* its results using Microwind simulator 74
Objectives 74
Pre-Lab 74
Lab Tasks 76
LAB # 12: [Open Ended Lab] 79
To implement the layout of data path components and *reproduce* the results using Microwind simulator 79
Objectives 79
Pre-Lab 79
Lab Tasks 80

LAB #1:
Introduction to DSCH and Microwind simulation tools and *follow* the steps to implement the design of CMOS inverter

Objective

Part 1

In this lab students will be introduced to a schematic based Electronic Design Automation (EDA) tool "DSCH" and the introduction will be accompanied with an implementation of Inverter at Gate level. The tool used in this lab is DSCH.

The goals for this Lab are:

- Familiarity and Hands on Example using the tool.
- Gate Level Design using the tool.
- Design Verification.
- Simulation of the design instill

Part 2

In this lab students will be introduced to a Layout based EDA tool "Microwind" and the introduction will be accompanied with analysis of MOS transistors. The tool used in this lab is Microwind. The goals for this Lab are:

- Familiarity and Hands on Example using the tool.
- Layout Design using the tool.
- Study of MOSFET Characteristics.
- Analog Simulation of MOSFETs.

Pre-Lab

A static CMOS gate is a combination of two networks, called the pull-up network (PUN) and the pull-down network (PDN) (Figure 1.2). The figure shows a generic N input logic gate where all inputs are distributed to both the pull-up and pull-down networks. The function of the PUN is to provide a connection between the output and VDD anytime the output of the logic gate is meant to be 1 (based on the inputs). Similarly, the function of the

PDN is to connect the output to VSS when the output of the logic gate is meant to be 0. The PUN and PDN networks are constructed in a mutually exclusive fashion such that one and only one of the networks is conducting in steady state.

In this way, once the transients have settled, a path always exists between VDD and the output F, realizing a high output ("one"), or, alternatively, between VSS and F for a low output ("zero"). This is equivalent to stating that the output node is always a low-impedance node in steady state. In this section, you will learn how to design an inverter using complementary CMOS family in DSCH and Microwind software.

The Inverter

An Inverter can be implemented using two MOSFETS i.e. one PMOS and one NMOS. Both MOSFETS are connected as shown in figure 1.1. V_{DD} is supplied to PMOS while the NMOS is grounded. Input is applied to the gate terminals of MOSFETs, and the output "F" is obtained from the common junctions as illustrated in Inverter circuit.

Inputs	Output
A	F
0	1
1	0

Table 1-1 Truth Table of Inverter

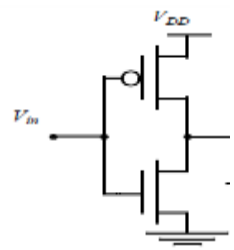


Figure 1-1 Transistor level Inverter

In-Lab

Part 1: Introduction to DSCH

The DSCH program is a logic editor and simulator. DSCH is used to validate the architecture of the logic circuit before the microelectronics design is started. DSCH provides a user-friendly environment for hierarchical logic design, and simulation with delay analysis, which allows the design and validation of complex logic structures. A key innovative feature is the possibility to estimate the power consumption of the circuit.

Implement the function of inverter ***x= x'*** by using NMOS and PMOS and then implement it on DSCH.

a) Turn on your PC and click on the DSCH icon.

b) DSCH will bring up two windows-the schematic drawing board and the symbol library.

c) The drawing board should be blank. The symbol library window presents you with choices for using predefined devices, such as basic gates, MOSFETS, LED and buttons.

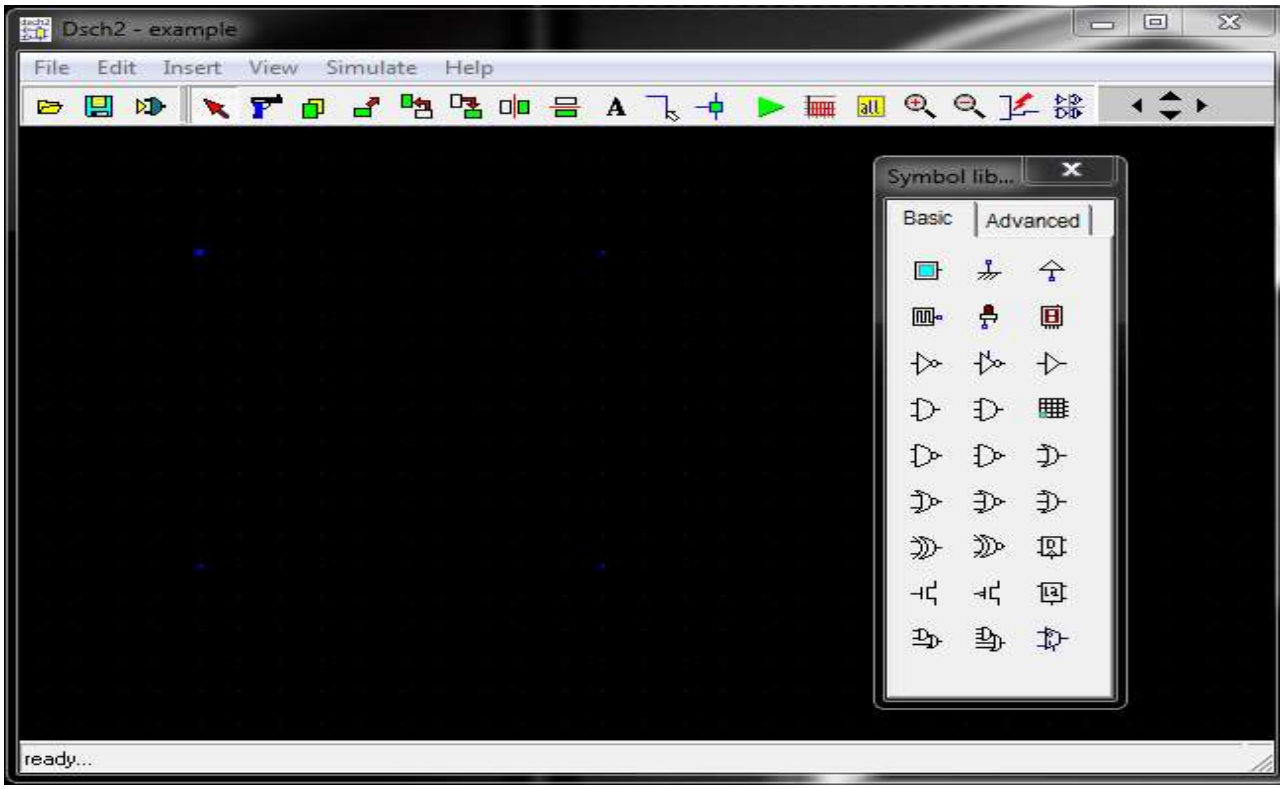


Figure 1-2 Main Window DSCH

TABLE 1- 2 DSCH ICON DETAIL

Icon	Description
	Cut
	Copy
	Move
	Rotate to Left
	Rotate to Right
	Flip
A	Add text
	Add a line
	Add a connector between lines

	Run Simulation
	Timing diagram
	Actual Size
	Zoom In
	Zoom Out
	View electrical net
	View symbol library

l

d) To select the foundry, use the command **File > Select Foundry**

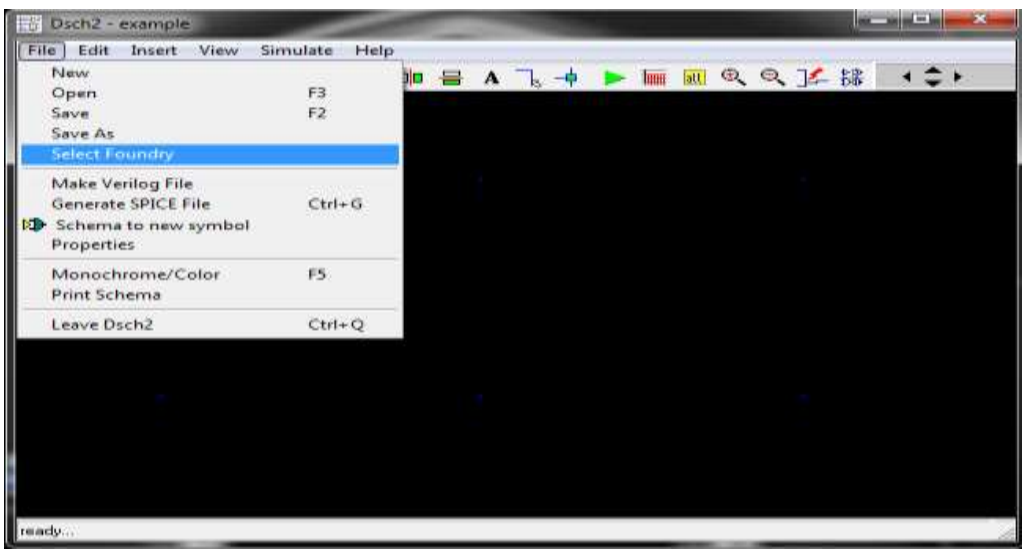


Figure 1-3 Select Foundry Window

e) We will be using **0.25-micron** process. For this, we will be selecting **"cmos025.tec"** file.

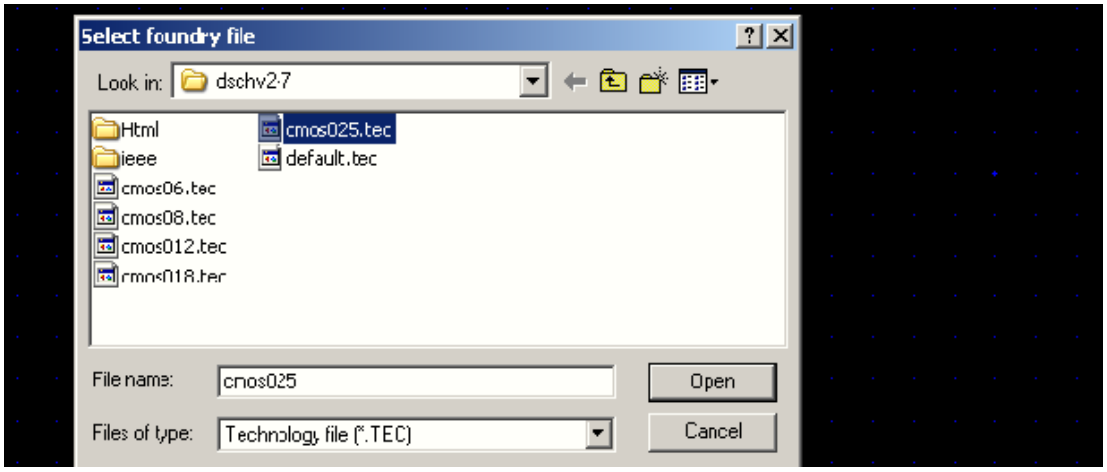


Figure 1-4 Select Foundry File

f) To create a new schematic. Delete the existing diagram (if any) by selecting the **Cut** command button from the icon menu bar and then select the whole diagram.

g) Choose the NMOS AND PMOS from the symbol library and arrange it as given in point

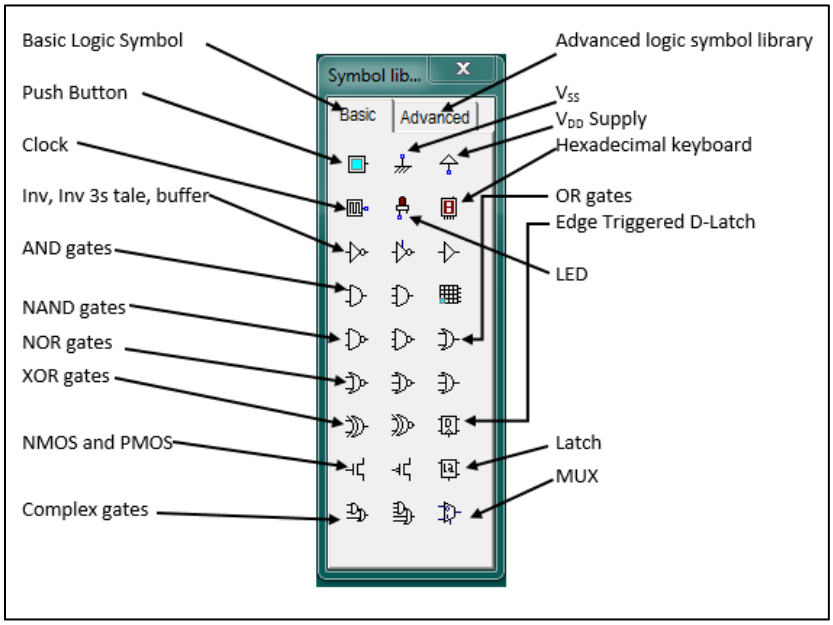


Figure 1-5 Symbol Library

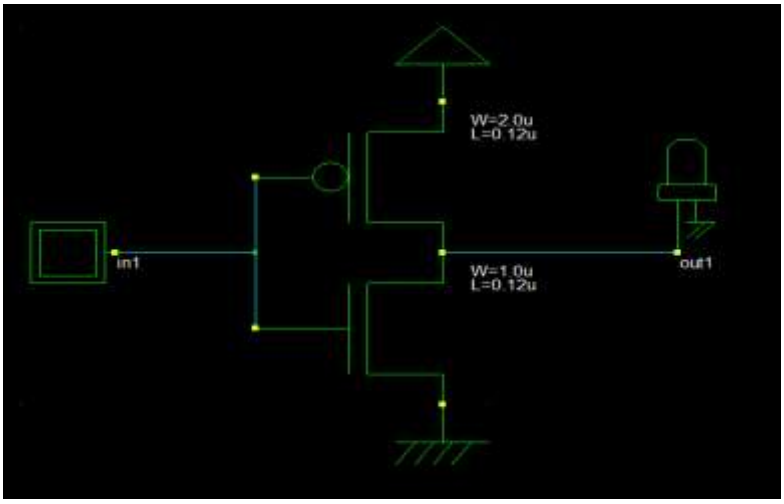


Figure 1-6 Schematic of Inverter

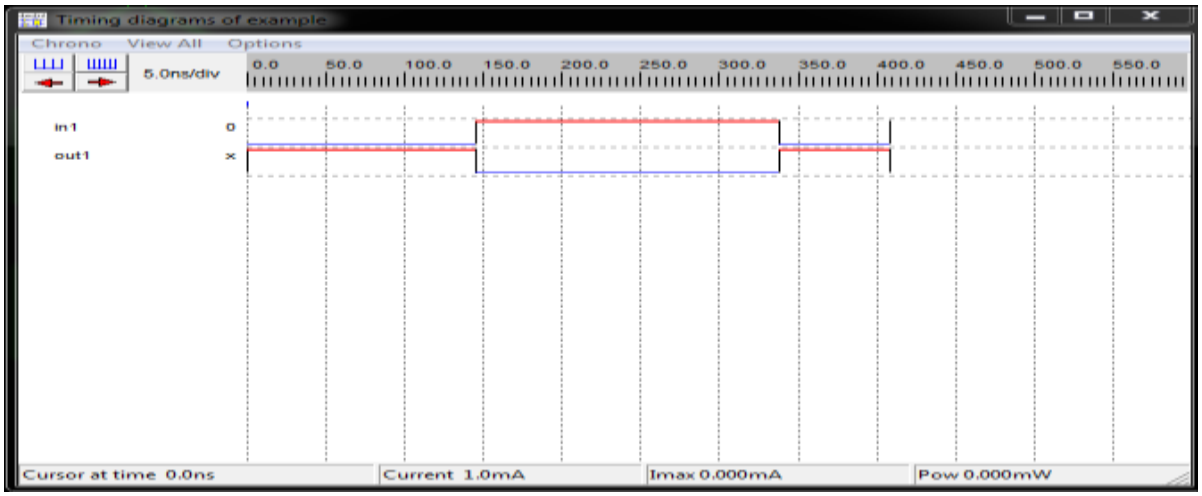


Figure 1-7 Timing Diagram of Inverter

Part 2: Introduction to Microwind

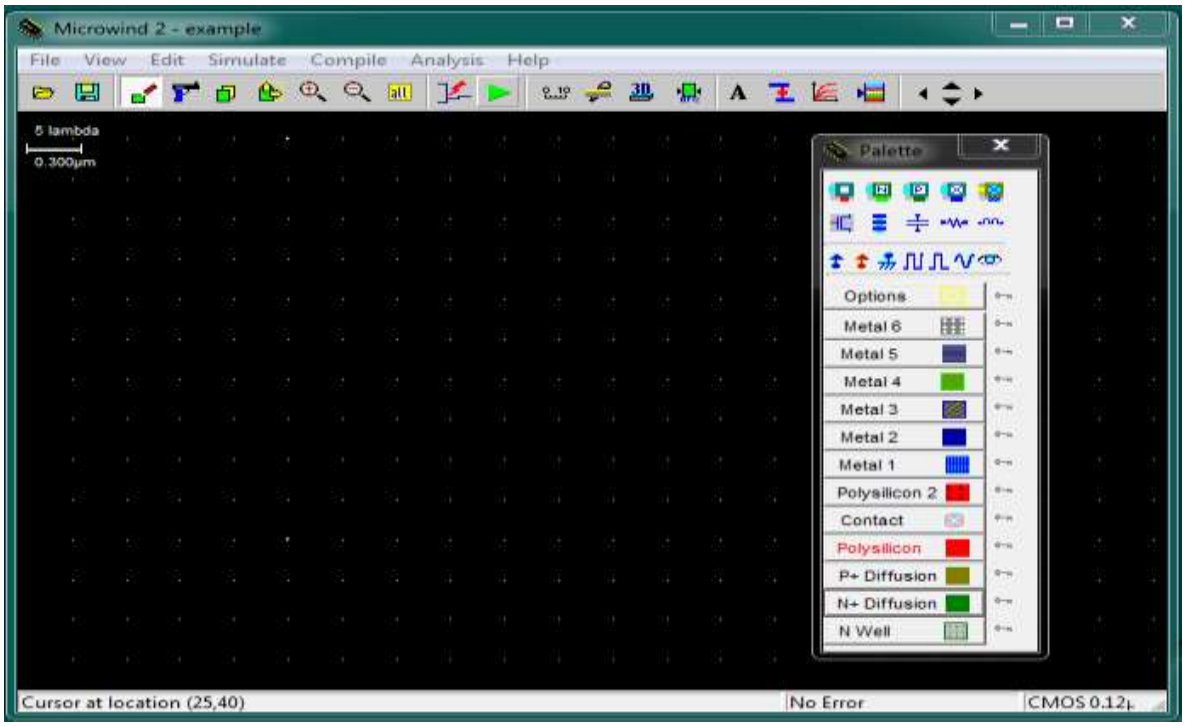


Figure 1-8 Main Window of Microwind

Icon	Description
	Cut
	Copy Element
	Stretch and move
	View All
	View electrical node
	Measure Distance
	Add text
	2-D Vertical cross section
	Process 3-D step
	Design rule checker

	Connect layers
	Simulate MOS characteristics
	Run Simulation
	Timing diagram
	Zoom In
	Zoom Out
	View electrical node
	Show Palette

Table 1-3 Microwind Icon Detail

To select the foundry using the command **File > Select Foundry**

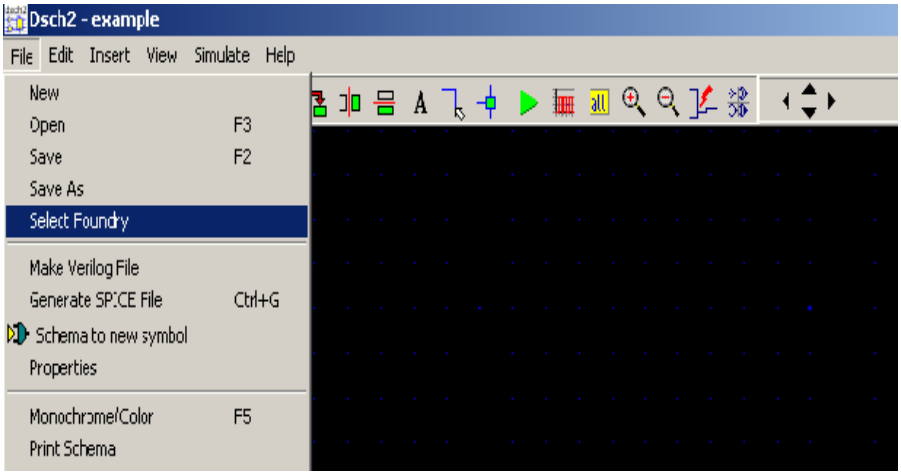


Figure 1-9 Select Foundry Window

We will be using 0.25-micron process by selecting **"cmos025.tec"** file. Click Open tab to continue.

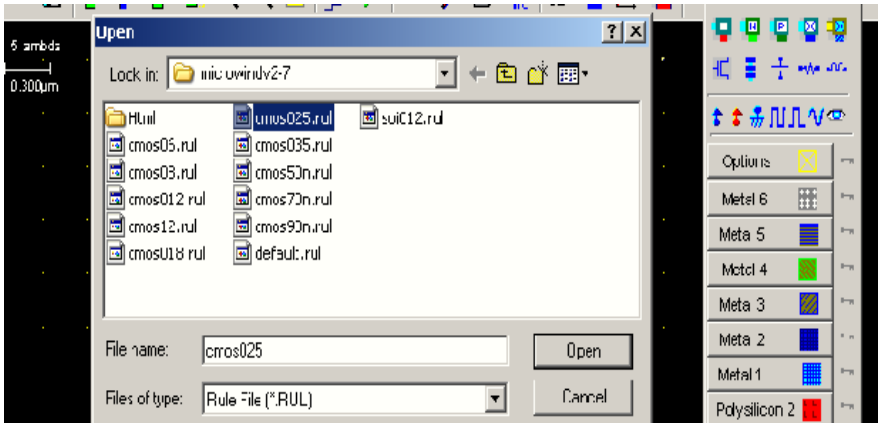


Figure 1-10 Foundry Selection in Microwind

To create an nMOS by using the **nMOS generator** button in the Palate

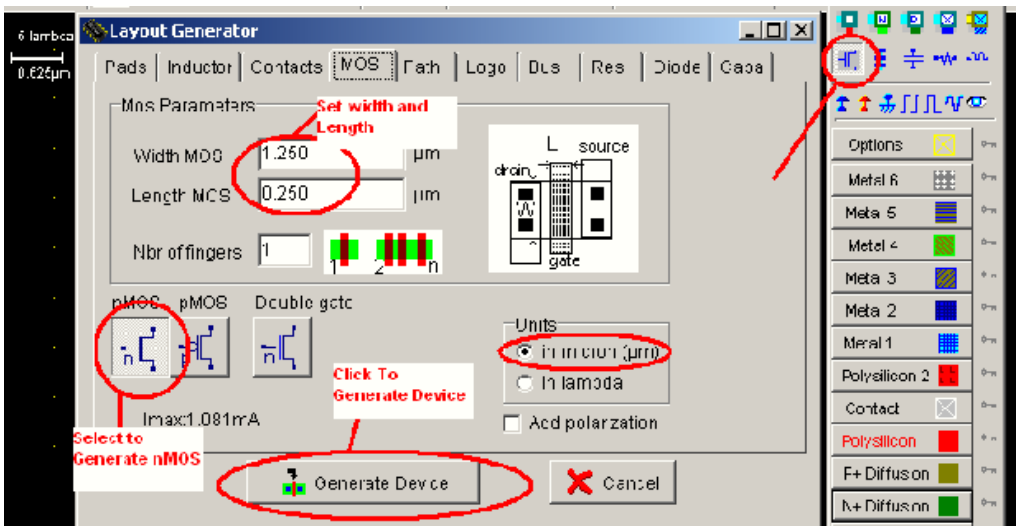


Figure 1-11 Layout Generator

You can set the width and length of MOS by typing in the fields **Width MOS and Length MOS** either in micron or in lambda units as indicated in the above figure.

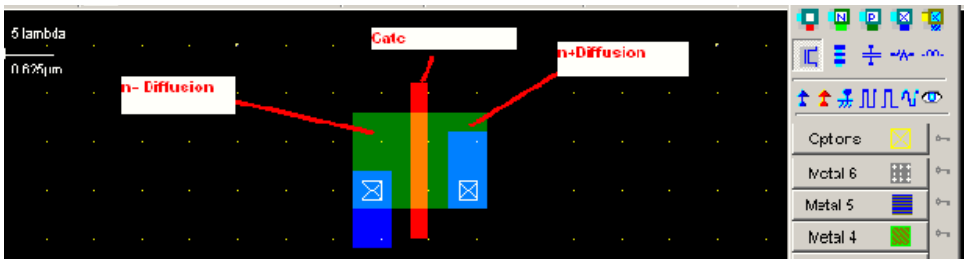


Figure 1-12 NMOS Symbol in Microwind

Apply the voltages and output node using the symbol buttons V_{DD}, Gnd, Add a Pulse, and Visible node in the Palate menu, as indicated in the following figure. You can use the **Stretch/Move** command button for these actions.

Now apply the V_{DD} to the n+ diffusion or drain terminal instead of Vss, run the Simulation again

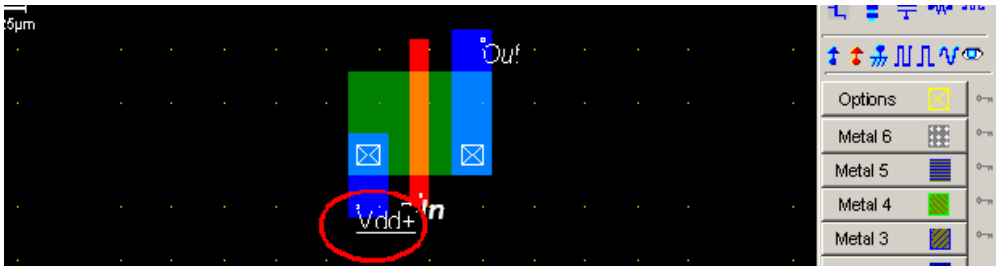


Figure 1-13 Applying Potential to NMOS

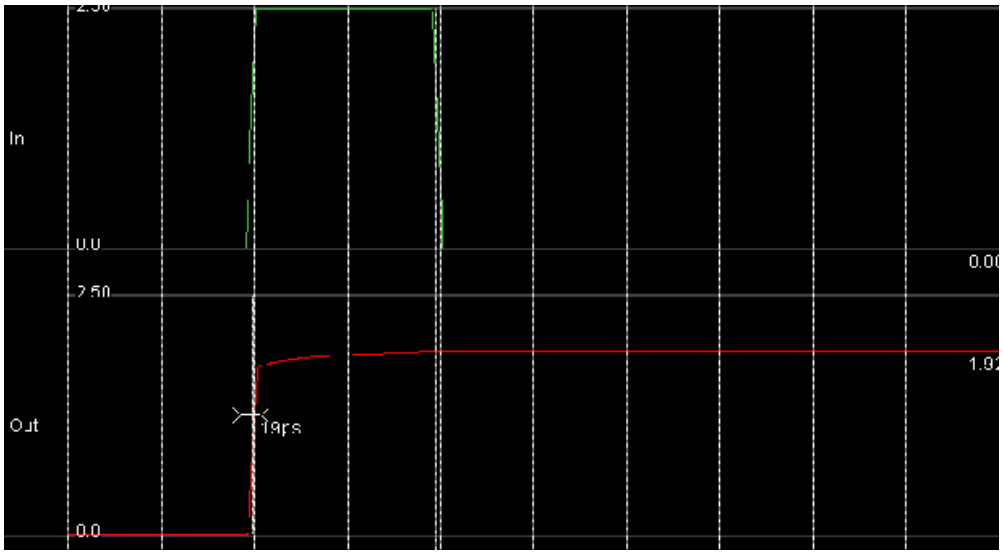


Figure 1-4 Analysis of NMOS in Microwind

Lab Tasks

Attach Layout and Results:

Task:

(a) *Follow* the steps given in this lab to implement the CMOS inverter on Microwind using 0.25um CMOS technology

(b) *Show* the timing diagram of the designed CMOS inverter.

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Rubric for Lab Assessment

Instructor Signature: _____ Date: _____

LAB # 2:

Part I: Introduction to VHDL and Altera Quartus tool and *follow* the steps to design static CMOS basic gates

Part II: To implement the design of static CMOS basic gates on Microwind and Altera Quartus tools and *reproduce* its results

Objective

In this lab students will design and implement the layouts of different CMOS gates, which includes NAND, NOR. The tool used in this lab is Microwind and Quartus. The goals for this Lab are:

- Design of CMOS NAND and NOR Gate.
- Layout Design using Microwind.
- Design using VHDL on Quartus
- Verify the simulation from timing diagram

Pre-Lab

Part I: Layout design of CMOS Gates:

NAND Gate

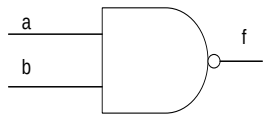
As per discussion and design on white board in the Lab, a NAND gate can be implemented using four FETS i.e. two PMOS and two NMOS as the inputs of the gate is two. Two PMOS are connected in parallel while two NMOS are connected in series, V_{DD} is supplied to the parallel combination of PMOS while the series combination of NMOS is grounded. Inputs a & b are applied to the gate terminals of all FETs, and the output f is obtained from the common junction of these series and parallel combinations as illustrated in NAND circuit under the heading of Design Diagram / Circuit.

NOR Gate

As per discussion and design on white board in the Lab, A NOR Gate can be implemented using fours FETS i.e two pFETs and two nFETs as the inputs of the gate is two. pFETs are connected in series while nFETs are connected in parallel, V_{DD} is supplied to the series combination of pFETs while the parallel combination of nFETs is grounded. Inputs a & b are applied to the gate terminals of all FETs, and the output f is obtained from the common junction of these parallel and series combinations as illustrated in NOR circuit under the heading of Design Diagram/Circuit.

Design Diagram / Circuit

a) Symbol, Truth Table and CMOS circuit of NAND Gate



a	b	f
0	0	1
0	1	1
1	0	1
1	1	0

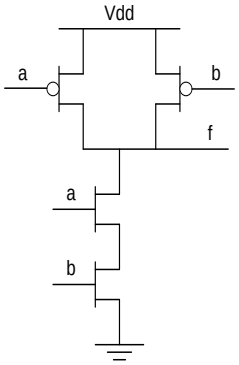
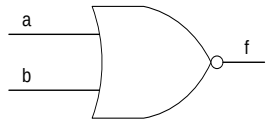


Figure 2-0-1 NAND Gate, Symbols and Truth Table

b) **Symbol, Truth Table and CMOS circuit of NOR Gate**



a	b	f
0	0	1
0	1	0
1	0	0
1	1	0

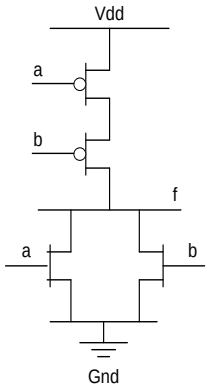


Figure 2-0-2 NOR Gate, Symbols and Truth Table

Part II: Familiarize yourself with VHDL and Quartus:

Introduction to Quartus:

Altera Quartus II is design software produced by Altera. Quartus II enables analysis and synthesis of HDL designs, which enables the developer to compile their designs, perform timing analysis, examine RTL diagrams, simulate a design's reaction to different stimuli, and configure the target device with the programmer. Quartus includes an implementation of VHDL and Verilog for hardware description, visual editing of logic circuits, and vector waveform simulation.

Introduction to VHDL

VHDL is a hardware description language. It described the behavior of an electronic circuit or system, from which the physical circuit or system can then be attained (implemented). VHDL stands for VHSIC Hardware Description Language. VHSIC is itself an abbreviation for Very High Speed Integrated Circuits, an initiative funded by US DOD in the 1980s that led to the creation of VHDL. VHDL is intended for circuit synthesis as well as circuit simulation. However, though VHDL is fully simulated, not all constructs are synthesizable.

A fundamental motivation to use VHDL (or its competitor, Verilog) is that VHDL is a standard, technology/vendor independent language, and is therefore portable and reusable. The two main immediate applications of VHDL are in the field of Programmable Logic Devices (including CPLDs and FPGAs) and ASICs. A final note regarding VHDL is that, contrary to regular computer programs which are sequential, its statements are inherently concurrent (parallel). In VHDL, only statements placed inside *a Process, Function or Procedure* are executed sequentially.

As mentioned above, one of the major utilities of VHDL is that it allows the synthesis of a circuit or system in a programmable device (PLD or FPGA) or in an ASIC. The steps followed during such a project are summarized in figure 1.1

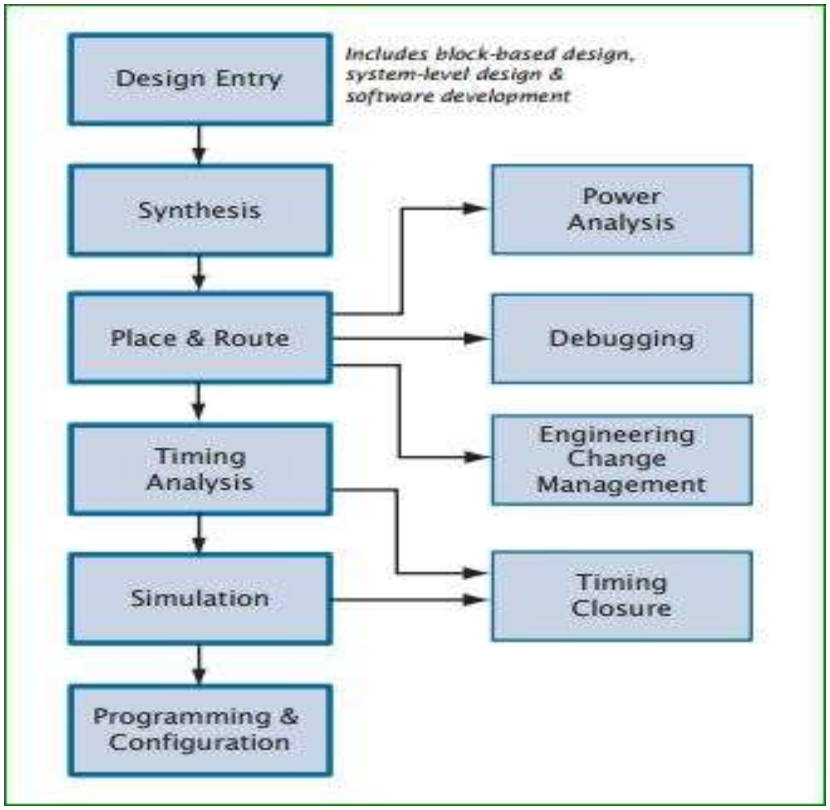


Figure 1.1 Quartus II Design Flow

Fundamental VHDL Units

Library declaration: Contains a list of all libraries to be used in the design. For example: ieee, std, work etc. Their declarations are as follows

Library ieee;

USE ieee.std_logic_1164.all;

USE ieee.std_logic_arith.all;

Entity:

VHDL files are basically divided into two parts, the entity and the architecture. The entity is basically where the circuits (in & out) ports are defined. There is a multitude of I/O ports available, but this lab will only deal with the two most usual ones, the INput and OUTput ports. (Other types of ports are for example the INOUT and BUFFER ports.) The entity of the circuit in figure 1.2 should look something like below. Please notice that comments in the code are made with a double-dash (--).

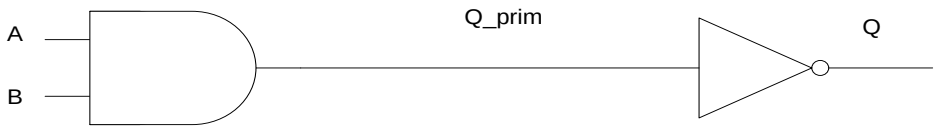


Figure 1.2 NAND gate using AND & NOT gate

```
ENTITY nandgate IS
PORT(
A: IN BIT;
B: IN BIT;
Q: OUT BIT -- Note! No ';' here!
);
END nandgate;
```

Now that we have defined the I/O interface to the rest of the world for the NAND gate, we should move on to the architecture of the circuit.

Architecture:

The entity told us nothing about how the circuit was implemented, this is taken care of by the architecture part of the VHDL code. The architecture of the NAND gate matching the entity above could then be written as something like this...

```
ARCHITECTURE dataflow OF nandgate IS
SIGNAL Q_prim: BIT;
BEGIN
Q_prim <= A AND B; -- The AND-operation.
Q <= NOT Q_prim; -- The inverter.
END dataflow;
```

Behavior modeling, Processes

We can write a behavior architecture body of an entity which describes the function in an abstract way. Such an architecture body includes only process statements.

Processes are used:

- For describing component behavior when they cannot be simply modeled as delay elements.
- To model systems at high levels of abstraction.

Process contains:

Conventional programming language constructs. A process is a sequentially executed block of code, which contains.

- Evaluating expressions.
- Conditional execution.
- Repeated execution.
- Subprogram calls.
- Variable assignments, e.g., x := y , which, unlike signal assignment, take effect immediately.
- if-then-else and loop statements to control flow, and
- Signal assignments to external signals.

Notes:

1. Signal assignment statements specify the new value and the time at which the signal is to acquire this value. The textual order of the concurrent signal assignment statements (CSAs) do NOT effect the results.

2. Processes contain sensitivity lists in which signals are listed, which determine when the process executes.

3. In reality, CSAs are also processes without the process, begin and end keywords.

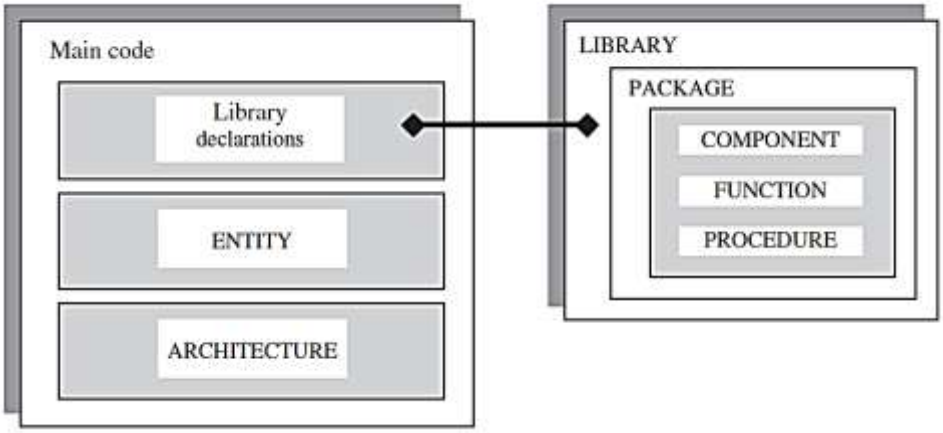
Structure modeling

Structural model: A description of a system in terms of the interconnection of its components, rather than a description of what each component does. A structural model does NOT describe how output events are computed in response to input events.

A VHDL structural description must possess:

- The ability to define the list of components.
- The definition of a set of signals to be used to interconnect them.
- The ability to uniquely label (distinguish between) multiple copies of the same component.

Creating your own library components:



Frequently used pieces of VHDL code are usually written in the form of COMPONENTS, FUNCTIONS, or PROCEDURES. Such codes are then placed inside a PACKAGE and compiled into the destination LIBRARY. The importance of this technique is that it allows code partitioning, code sharing, and code reuse.

We start by describing the structure of a PACKAGE. Besides COMPONENTS, FUNCTIONS, and PROCEDURES, it can also contain TYPE and CONSTATNT definitions, among others. Its syntax is presented below

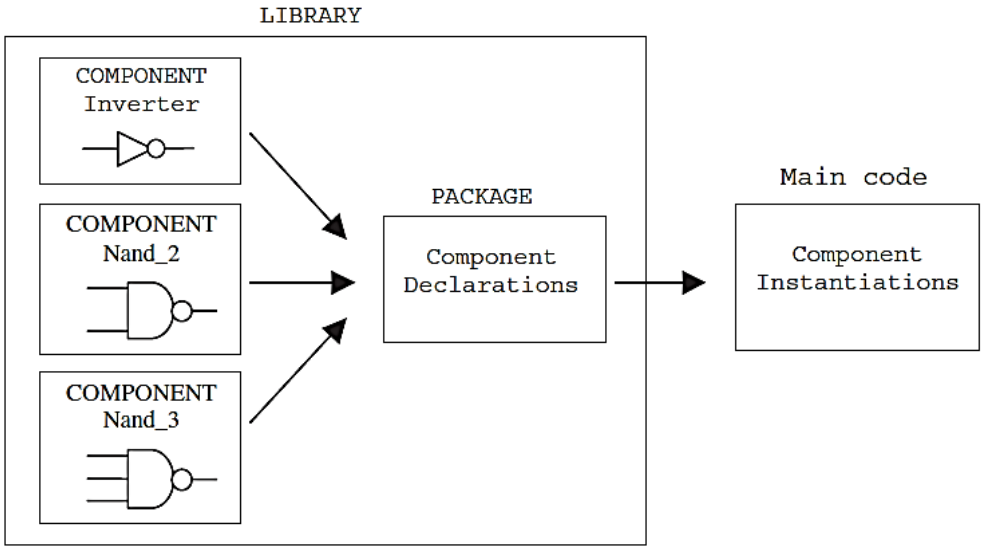
```
PACKAGE package_name IS
(declarations)
END package_name;
[ PACKAGE BODY package_name IS
      (FUNCTION and PROCEDURE descriptions)
END package_name; ]
```

The syntax is composed of two parts: PACKAGE and PACKAGE BODY. The first part is mandatory and contains all declarations, while the second part is necessary only when one or more subprograms (FUNCTION or PROCEDURE) are declared in the upper part, in which case it must contain the descriptions (bodies) of the subprograms. PACKAGE and PACKAGE BODY must have the same name.

The declaration list can contain the following COMPONENT, FUNCTION, PROCEDURE, TYPE, CONSTANT etc.

Example:

The example below shows a how to use PACKAGE called my_components. It contains three components *inverter, nand_2 and nand_3* in it.



----------Code for myinverter.vhd

```
library IEEE;
use ieee.std_logic_1164.all;
entity myinverter is
port(
          inv_in: in std_logic;
          inv_out: out std_logic
);
end myinverter;
architecture bhv of myinverter is
begin
     inv_out <= not inv_in;
end bhv;
```

----------Code for nand_2.vhd

```
library IEEE;
use ieee.std_logic_1164.all;
entity nand_2 is
port(
          nand2_in1: in std_logic;
          nand2_in2: in std_logic;
          nand2_out: out std_logic
);
end nand_2;
architecture nand_2 of nand_2 is
begin
nand2_out <= NOT (nand2_in1 AND nand2_in2);
end nand_2;
```

----------Code for nand_3.vhd

```
library IEEE;
use ieee.std_logic_1164.all;
entity nand_3 is
port(
          nand3_in1: in std_logic;
          nand3_in2: in std_logic;
          nand3_in3: in std_logic;
          nand3_out: out std_logic
);
end nand_3;
architecture nand_3 of nand_3 is
begin
     nand3_out <= NOT (nand3_in1 AND nand3_in2 AND nand3_in3);
end nand_3;
```

----------Code for my_components.vhd

```
library ieee;
use ieee.std_logic_1164.all;

package my_components is
component myinverter is
port( inv_in: in std_logic; inv_out: out std_logic);
end component;

component nand_2 is
port  (    nand2_in1: in std_logic;
                nand2_in2: in std_logic;
                nand2_out: out std_logic
          );
end component;

component nand_3 is
port(
          nand3_in1: in std_logic;
          nand3_in2: in std_logic;
          nand3_in3: in std_logic;
          nand3_out: out std_logic
);
end component;
end my_components;
```

A. Steps to Create a New Project:

1. **Start the Quartus II software.**

   From the Windows Start Menu, select:

   All Programs → Other Apps →Altera → Quartus II 13.1 → Quartus II 13.1 (32-Bit)

2. **Start the New Project Wizard.**

If the opening splash screen is displayed, select: **Create a New Project (New Project Wizard)**, otherwise from the Quartus II Menu Bar select: **File** → **New Project Wizard**.

3. **Select the Working Directory and Project Name.**

Working Directory	H:\Altera_Training\Lab1
Project Name	Lab1
Top-Level Design Entity	Lab1

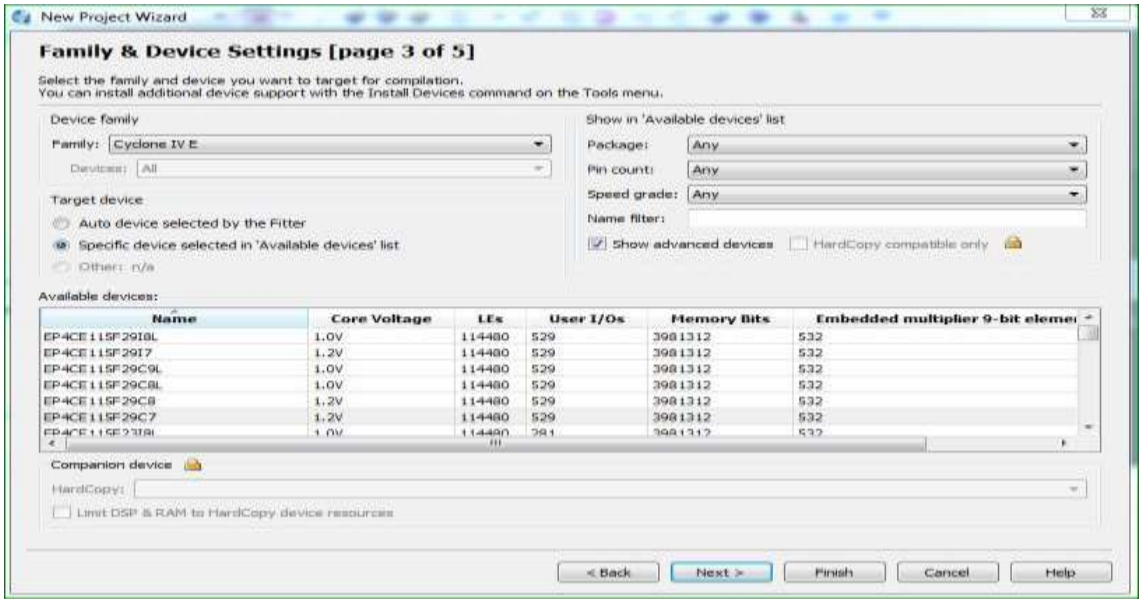
Click **Next** to advance to page 2 of the New Project Wizard.

*Note: A window may pop up stating that the chosen working directory does not exist. Click **Yes** to create it.*

4. Click **Next** again as we will not be adding any preexisting design files at this time.

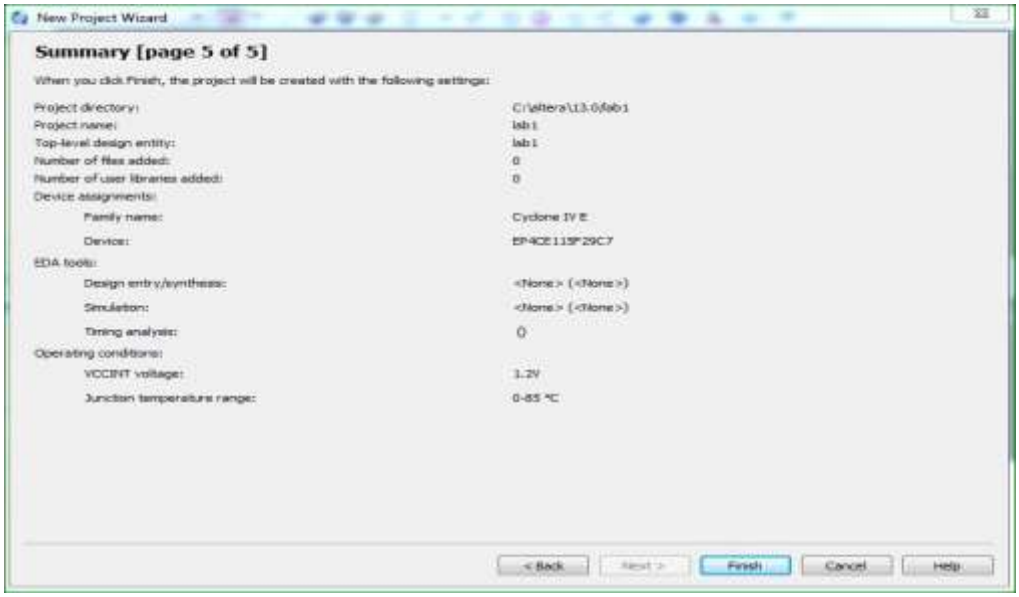
5. **Select the family and Device Settings.**

From the pull-down menu labeled **Family**, select **Cyclone IV**.In the list of available devices, select **EP4CE115F29C7N**. Click **Next.**



6. Click **Next** again as we will not be using any third-party EDA tools

7. Click Finish to complete the New Project Wizard

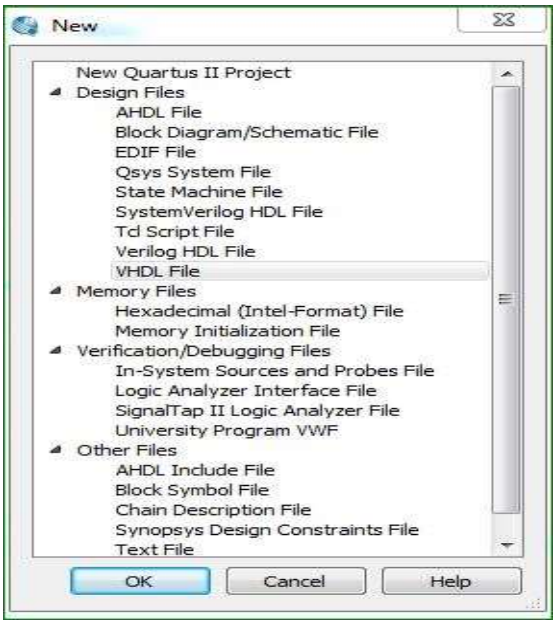


B. Create, Add, and Compile Design Files

1. **Create a new Design File.**

   Select: **File → New** from the Menu Bar.

   Select: **VHDL File** from the **Design Files** list and click **OK**.



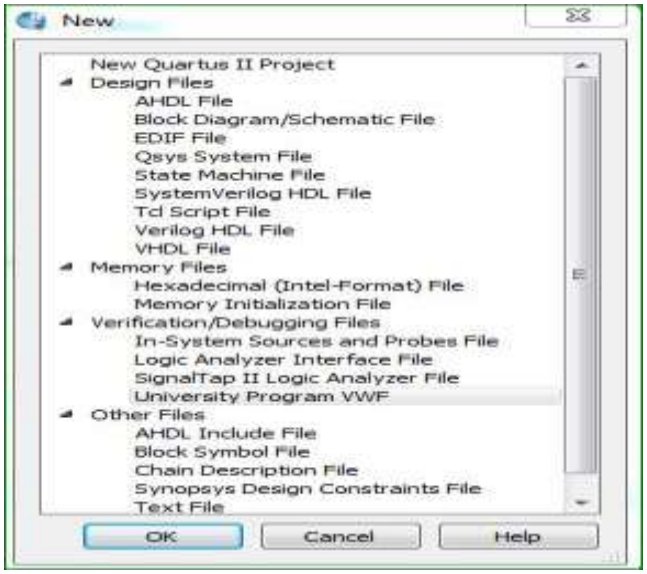
2. Copy and paste the code given in Example into your new VHDL file, then save it by selecting **File →Save**. Name the file **my_components** and click **Save** in the **Save As** dialog box.

C. Simulate Design using Quartus II

1. Create a Vector Waveform File (vwf)

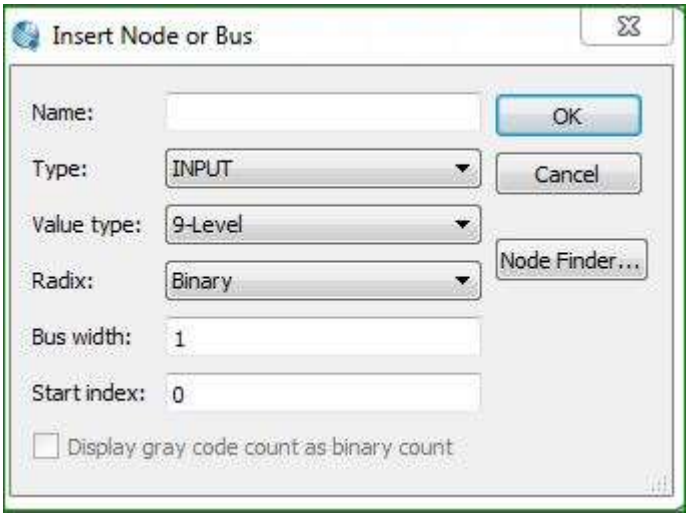
Click the **New File** icon on the Menu Bar

Select **Vector Waveform File** under the **Verification/Debugging Files** heading.



2. Add Signals to the Waveform

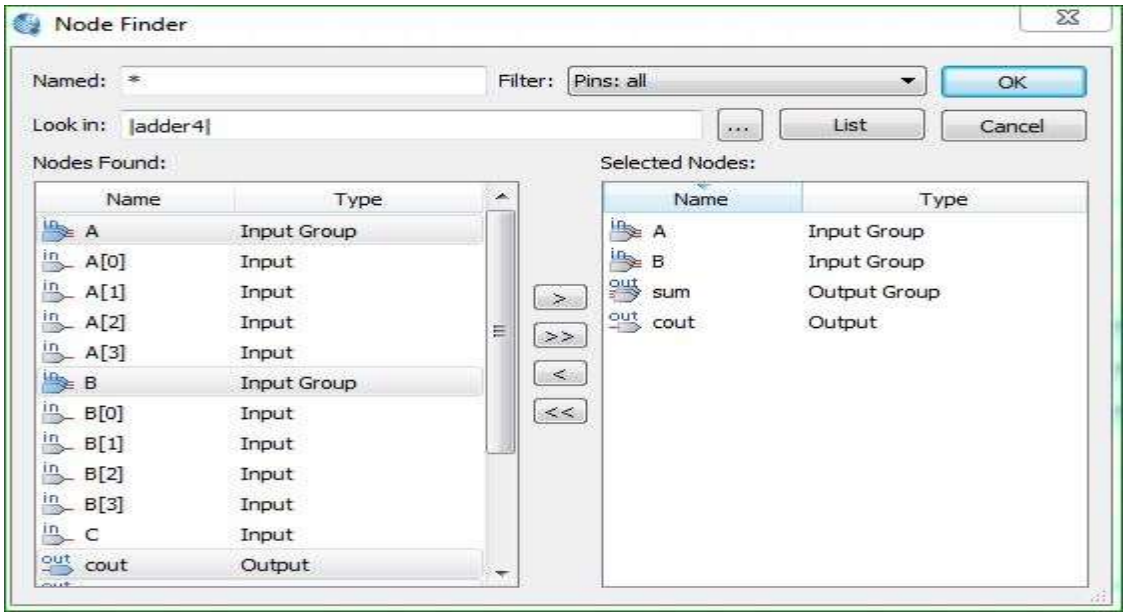
Select **Edit → Insert → Insert Node or Bus…** to open the **Insert Node or Bus** window.



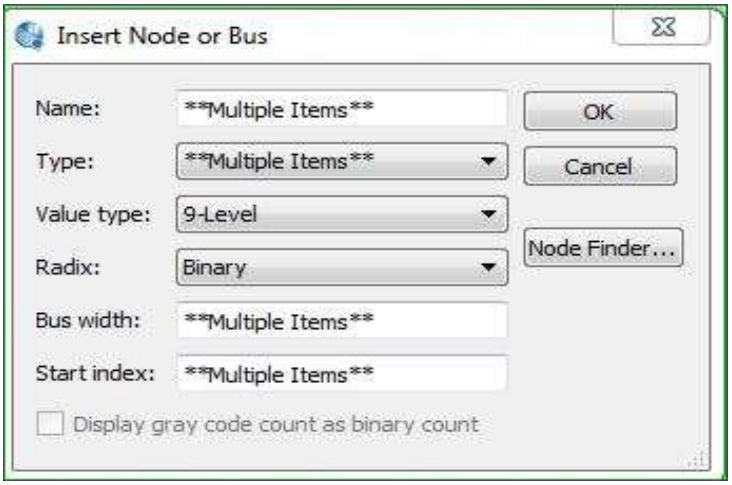
3. Click the **Node Finder…** button in the **Insert Node or Bus** window to open the **Node Finder** window.

From the **Filter** drop-down menu, select **Pins: all** and click the **List** button. This will display all the inputs and outputs to the circuit.

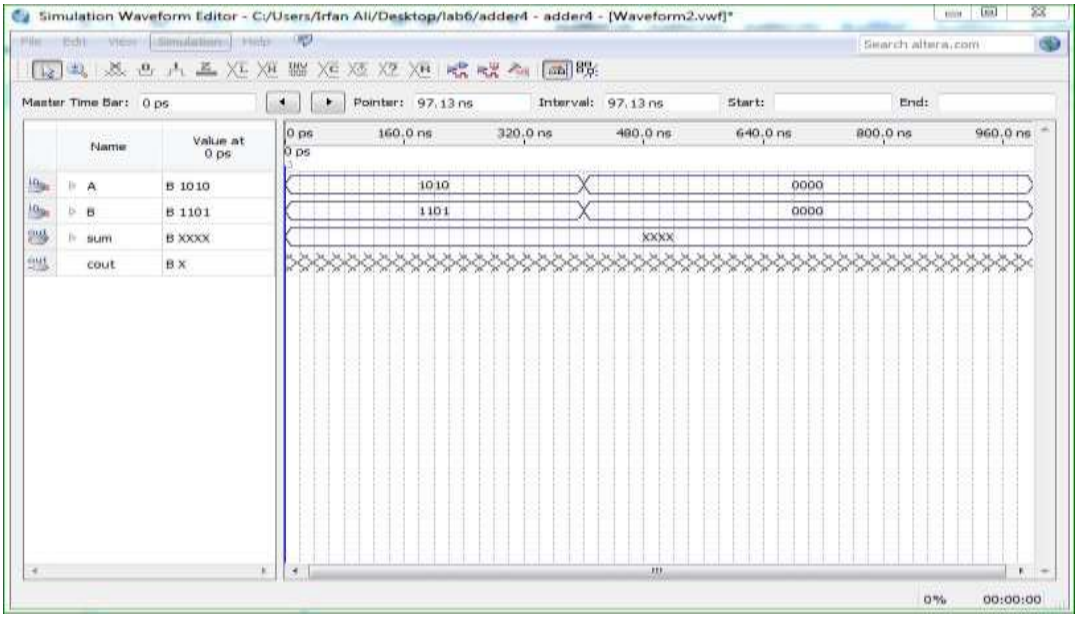
Highlight all the the **Input** and **Output** in the **Nodes Finder** window and click the right arrow to select them. Then click **OK** to close the **Node Finder** window.



4. In the **Insert Node or Bus** window, change **Radix** to **Binary**. This will make it easier for us to enter values and interpret the results.



5. Input waveforms can be drawn in different ways. The most straightforward way is to indicate a specific time range and specify the value of a signal. To illustrate this approach, click the mouse on the A waveform near the 0-ns point and then drag the mouse to the 400-ns point. The selected time interval will be highlighted in blue. Press **Ctrl + Alt + B** to open the **Arbitrary Value** window to enter the values manually.



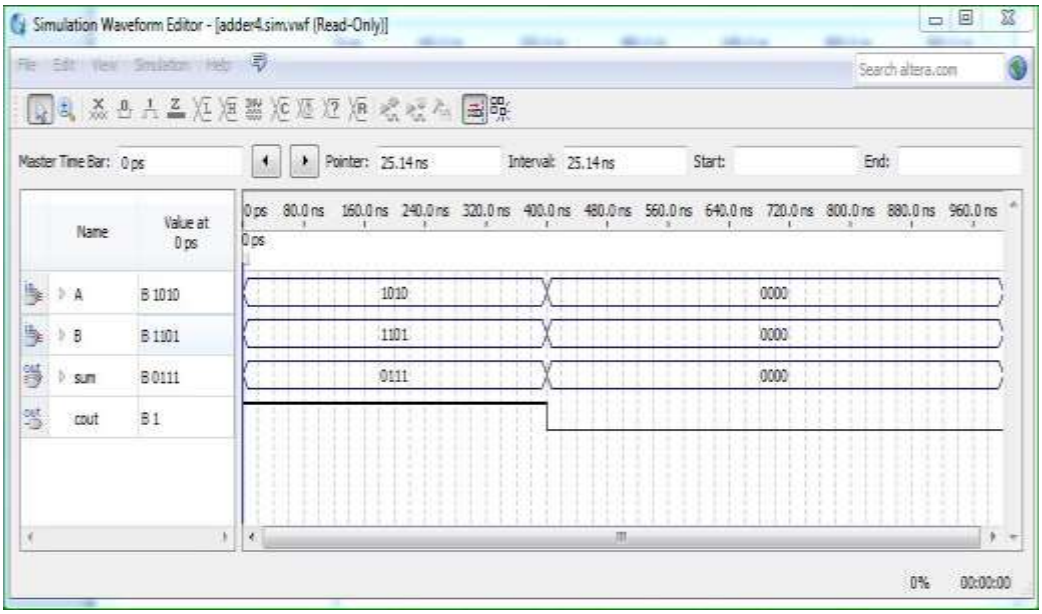
Leave **output** with a value of undefined (X), as values for this will be generated when the simulation runs. Click the **Save** icon on the Menu bar and save the vwf file with the filename **my_components.vwf**.

6. In the Main window, select *Simulation | Options* and then select *Quartus II Simulator*. Select *OK*.

7. In the Main window, select *Simulation* and then select *Run Functional Simulation*.

8. Now you should see your simulation output with the outputs defined.

Note: The file will indicate "read-only" meaning you can't edit it.



Lab Tasks

Task I:

(a) *Follow* the steps given in this experiment to implement the layout of 3-input NAND gate using 0.25um CMOS technology on Microwind simulator.

Note: Use MOS Generator to generate layout of NMOS and PMOS.

(b) *Show* the result using Microwind simulator

Task II:

(a) Write an HDL code for 3-input NAND module and *reproduce* its vector waveform on Quartus.

(b) Compare the results of Task I (b) and Task II (a).

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____

LAB # 3:
To *explain* the layout of static CMOS basic gates using bubble pushing and implement it on Microwind simulator

Objective

In this lab students will design and implement the layouts of different CMOS gates, which includes NAND, NOR. The tool used in this lab is Microwind. The goals for this Lab are:

- Introduction to the bubble pushing
- Converting Boolean function to the NAND gate only
- Converting Boolean function to the NOR gate only
- Design the layout on Microwind and verify by timing diagram

Pre-Lab

Bubble pushing

Static CMOS circuits with complementary nMOS pulldown and pMOS pull-up networks are used for the vast majority of logic gates in integrated circuits. They have good noise margins, and are fast, low power, insensitive to device variations, easy to design, widely supported by CAD tools, and readily available in standard cell libraries. When noise does exceed the margins, the gate delay increases because of the glitch, but the gate eventually will settle to the correct answer. Most design teams now use static CMOS exclusively for combinational logic. This section begins with a number of techniques for optimizing static CMOS circuits.

Designers accustomed to AND and OR functions must learn to think in terms of NAND and NOR to take advantage of static CMOS. In manual circuit design, this is often done through bubble pushing. Compound gates are particularly useful to perform complex functions with relatively low logical efforts. When a particular input is known to be latest, the gate can be optimized to favor that input.

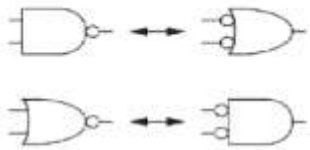


Figure 3-1 Bubble Pushing

Now we will design a circuit to compute $F = AB + CD$ using NANDs only. By inspection, the circuit consists of two ANDs and an OR, shown in Figure 3.2(a). In Figure 3.3(b), the ANDs and ORs are converted to basic CMOS stages. In Figure 3.2(c and d), bubble pushing is used to simplify the logic to three NANDs.

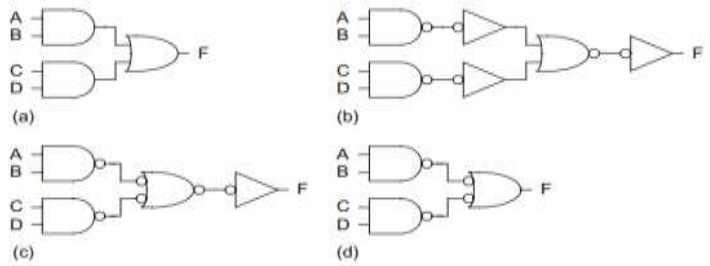


Figure 3-2

Lab Tasks

Task I:

Attach Layout and results

(a) Proceed with the implementation of the layout of the function F =AB +CD using 0.25um CMOS technology on Microwind based on:

- NAND gate
- NOR gate

(b) Compare both layouts to *explain* which transformation is optimal according to the transistors count.

(c) *Show* the results using Microwind simulator

Task II:

(a) Proceed with the implementation of the layout of the function F = ((A+B+C). D)' using 0.25um CMOS technology on Microwind using

- NAND gate only
- NOR gate only

(b) Compare both layouts to *explain* which transformation is optimal according to the transistors count.

(c) *Show* the results using Microwind simulator

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____

LAB # 4:
To *explain* the layout of 2 x 1 multiplexer using Transmission gate and Tristate buffer to implement its schematic on Microwind simulator and VHDL code on Quartus simulation tool to generate its vector waveform

Objectives

In this lab students will design and implement the layouts of multiplexer, which includes transmission gate and tristate buffer. The tool used in this lab is Microwind and Quartus. The goals for this Lab are:

- Introduction to transmission gate and tristate buffer
- To design the layout of the 2 x 1 multiplexer on Microwind and Quartus
- To design the layout of the 2 x 1 multiplexer on Microwind and Quartus
- Verify result by timing diagram and vector waveform

Pre-Lab

Transmission gate

A transmission gate, or analog switch, is defined as an electronic element that will selectively block or pass a signal level from the input to the output. This solid-state switch is comprised of a pMOS transistor and nMOS transistor. The control gates are biased in a complementary manner so that both transistors are either on or off.

When the voltage on node A is a Logic 1, the complementary Logic 0 is applied to node active-low A, allowing both transistors to conduct and pass the signal at IN to OUT. When the voltage on node active-low A is a Logic 0, the complementary Logic 1 is applied to node A, turning both transistors off and forcing a high-impedance condition on both the IN and OUT nodes. This high-impedance condition represents the third "state" (high, low, or high-Z).

The schematic diagram (Figure 4-1) includes the arbitrary labels for IN and OUT, as the circuit will operate in an identical manner if those labels were reversed. This design provides true bidirectional connectivity without degradation of the input signal.

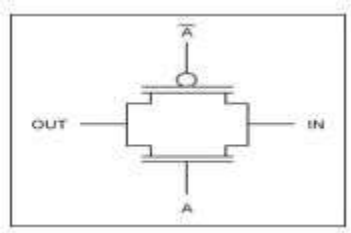


Figure 4-1 Transmission gate

Tristate buffer

Figure 4-3(d) shows symbols for a *tristate inverter*. When the enable input EN is 1, the output *Y* equals the input *A'*, just as in an ordinary inverter. When the enable is 0, *Y* is left floating (a 'Z' value). This is summarized in Table 4-2. Sometimes both true and complementary enable signals EN and EN are drawn explicitly, while sometimes only EN is shown. The transmission gate has the same truth table as a tristate buffer. It only requires two transistors, but it is a *non-restoring* circuit. If the input is noisy or otherwise degraded, the output will receive the same noise.

Table 4-2 Tri State Inverter Truth Table

EN / EN	***A***	***Y***
0 / 1	0	Z
0 / 1	1	Z
1 / 0	0	1
1 / 0	1	0

Figure 4-3(a) shows a *tristate inverter*. The output is actively driven from *VDD* or GND, so it is a restoring logic gate. Unlike any of the gates considered so far, the tristate inverter does not obey the conduction complements rule because it allows the output to float under certain input combinations. When EN is 0 (Figure 4-3(b)), both enable transistors are OFF, leaving the output floating. When EN is 1 (Figure 4-3(c)), both enable transistors are ON. They are conceptually removed from the circuit, leaving a simple inverter. Figure 4-3(d) shows symbols for the tristate inverter. The complementary enable signal can be generated internally or can be routed to the cell explicitly. A tristate buffer can be built as an ordinary inverter followed by a tristate inverter.

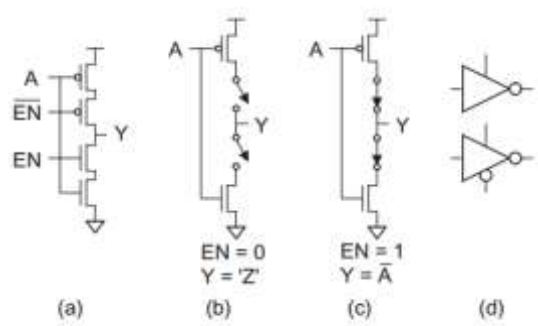


Figure 4-3 Tri State Inverter

Multiplexers are key components in CMOS memory elements and data manipulation structures. A multiplexer chooses the output from among several inputs based on a select signal. A 2-input, or 2:1 multiplexer, chooses input D0 when the select is 0 and input D1 when the select is 1. Two transmission gates can be tied together to form a compact 2-input multiplexer, as shown in Figure 4-4. The select and its complement enable exactly one of the two transmission gates at any given time. The complementary select S is often not drawn in the symbol.

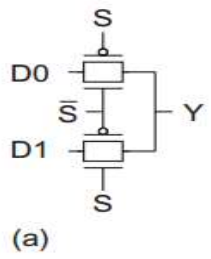


Figure 4-4 2x1 MUX with transmission gate

Again, the transmission gates produce a non-restoring multiplexer. You could build a restoring, inverting multiplexer out of gates in several ways. One is the compound gate connected as shown in Figure 4-5(a). Another is to gang together two tristate inverters, as shown in Figure 4-5 (b). Notice that the schematics of these two approaches are nearly identical, save that the pull-up network has been slightly simplified and permuted in Figure 4-5(b). This is possible because the select and its complement are mutually exclusive. The tristate approach is slightly more compact and faster because it requires less internal wire. Again, if the complementary select is generated within the cell, it is omitted from the symbol (Figure 4-5(c)). Larger multiplexers can be built from multiple 2-input multiplexers or by directly ganging together several tristate.

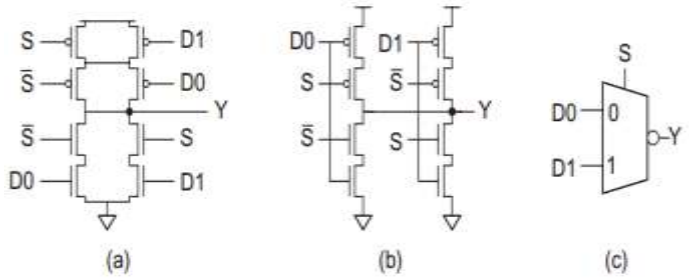


Figure 4-5 2x1 MUX with Tri State Buffer

Lab Tasks

Task I:

(a) *Follow* the schematic diagram of 2x1 multiplexer designed with Tri State Inverter to implement its layout using 0.25um CMOS technology.

Note: Use MOS Generator to generate layout of NMOS and PMOS.

(b) *Show* the result on Microwind simulator

Task II:

(a) Write VHDL code for 4x1 multiplexer module called mux4 with inputs S1:0, D0, D1, D2, D3 and output Y.

(b) Generate its vector waveform using Quartus.

Task III:

(a) Write a structural module to compute Y = AB + BC + ABC using multiplexer logic. Note: Use the 4:1 multiplexer from Task II.

(b) *Reproduce* its vector waveform using Quartus.

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____

LAB # 5:

To analyze λ - based design rules of CMOS transistor in Microwind, design stick diagrams of basic static CMOS gates and reproduce the results using Microwind simulator

Objective

In this lab students will learn the concepts of lambda-based design rules of CMOS circuits. They will also design stick diagram. Different design Parameter's effects like transistor sizing, cell width, height and area will be analyzed and spacing of metal, polysilicon and contacts will be measured etc. The tool used in this lab is Microwind.

Goals of this lab are:

- Analyze the lambda-based design rules of CMOS circuits
- Design / Sketch stick diagram
- Design the layout of given CMOS

Pre-Lab

Layout design rules:

Layout design rules describe how small features can be and how closely they can be reliably packed in a manufacturing process. Industrial design rules are usually specified in microns. This makes migrating from one process to a more advanced process or a different foundry's process difficult because not all rules scale in the same way. Mead and Conway popularized scalable design rules based on a single parameter, λ , that characterizes the resolution of the process λ- is generally half of the minimum drawn transistor channel length. This length is the distance between the source and drain of a transistor and is set by the minimum width of a polysilicon wire. Designers often describe a process by its feature size. Feature size refers to minimum transistor length, so λ is half the feature size.

A conservative but easy-to-use set of design rules for layouts with two metal layers in an n-well process is as follows:

- Metal and diffusion have minimum width and spacing of 4 λ.
- Contacts are 2 λ x 2 λ and must be surrounded by 1 λ on the layers above and below.
- Polysilicon uses a width of 2 λ.
- Polysilicon overlaps diffusion by 2 λ where a transistor is desired and has a spacing of 1 λ away where no transistor is desired.
- Polysilicon and contacts have a spacing of 3 λ from other polysilicon or contacts.

- N-well surrounds pMOS transistors by 6 λ and avoids nMOS transistors by 6 λ.

Transistor dimensions are often specified by their Width/Length (W/L) ratio. For example, the nMOS transistor in Figure 5.11 formed where polysilicon crosses n-diffusion has a W/L of 4/2. In a 0.6 um process, this corresponds to an actual width of 1.2 um and a length of 0.6 um. pMOS transistors are often wider than nMOS transistors because holes move more slowly than electrons, so the transistor has to be wider to deliver the same current.

Design/ Diagram/Circuit

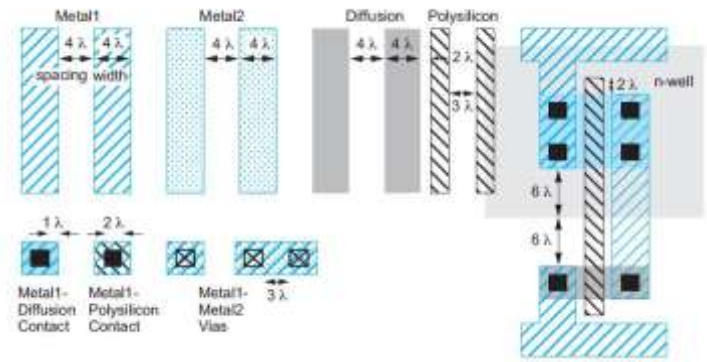


Figure 5.0-11 Simplified λ-based design rules

Figure 5.12 shows a unit inverter layout with a unit nMOS transistor and a double-sized pMOS transistor. Figure 5.13 shows a schematic for the inverter annotated with Width/ Length for each transistor. In digital systems, transistors are typically chosen to have the minimum possible length because short-channel transistors are faster, smaller, and consume less power. Figure 5.14 shows a shorthand we will often use, specifying multiples of unit width and assuming minimum length.

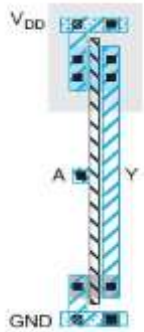


Figure 5.12

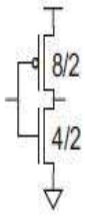


Figure 5.13

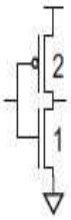


Figure 5.14

Unit Inverter

Stick Diagrams:

Designers need fast ways to plan cells and estimate area before committing to a full layout. Stick diagrams are easy to draw because they do not need to be drawn to scale. Figure 5.21 and the inside front cover show stick diagrams for an inverter and a 3-input NAND gatE

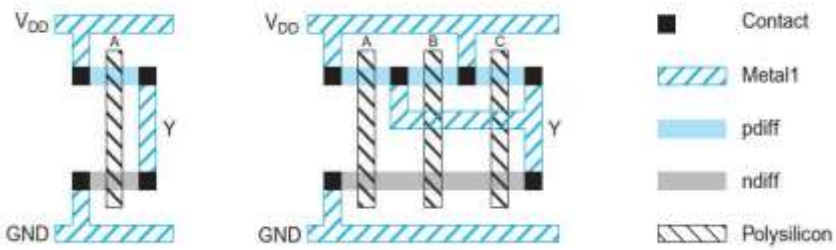
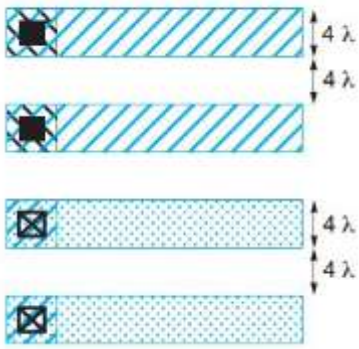


Figure 5.21 Stick diagrams of inverter and 3-input NAND gate

We define a routing track as enough space to place a wire and the required spacing to the next wire. If our wires have a width of 4 λ and a spacing of 4 λ to the next wire, the track pitch is 8 λ, as shown in Figure 5.22. This pitch also leaves room for a transistor to be placed between the wires Figure 5.23.



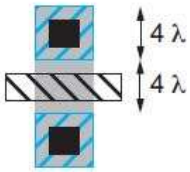


Figure 5.22 *Figure 5.23*

Pitch of routing tracks

Therefore, it is reasonable to estimate the height and width of a cell by counting the number of metal tracks and multiplying by 8 λ. A slight complication is the required spacing of 12 λ between nMOS and pMOS transistors set by the well, as shown in Figure 5.24.

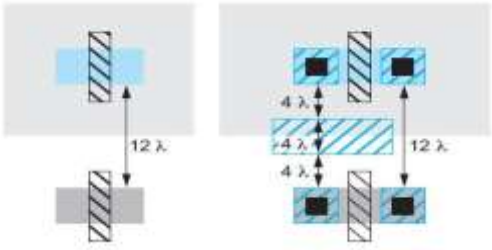


Figure 5.24 Spacing between nMOS and pMOS transistors

Figure 5.25 shows how to count tracks to estimate the size of a 3-input NAND. There are four vertical wire tracks, multiplied by 8 λ per track to give a cell width of 32 λ. There are five horizontal tracks, giving a cell height of 40 λ. Even though the horizontal tracks are not drawn to scale, they are still easy to count.

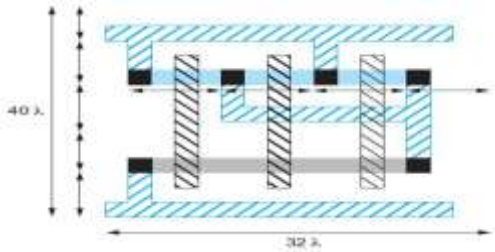


Figure 5.25 3-input NAND gate area estimation

Lab Tasks

Task-I:

Consider the design of a CMOS compound OR-AND-INVERT (OAI21) gate computing the function F given below and perform the following tasks:

$$F = ((A + B) \cdot c)'.$$

a) *Follow* the design rules to sketch stick diagram for Function **'F'**.

b) *Follow* the design rules to estimate the area (Cell width and height) from the stick diagram

c) *Reproduce* the layout on Mircowind tool using unit-sized transistors

d) Compare the layout size of implemented design with the estimated area

Task-II:

a) Consider the design of a CMOS compound OR-OR-AND-INVERT (OAI22) gate computing the Function **'F'** given below and perform the following tasks:

$$F = ((A + B) \cdot (C + D))'.$$

b) *Follow* the design rules to sketch its stick diagram

c) *Follow* the design rules to estimate the area (Cell width and height) from the stick diagram

d) *Reproduce* the layout on Mircowind tool

e) Compare the layout size of implemented design with the estimated area

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____

LAB # 6:
To analyze the delay of CMOS gates in Microwind and reproduce the results using Microwind simulator

Objective

In this lab students will learn the concepts of various kinds of delays like propagation delay, contamination delay, parasitic delay etc and also about logical effort in CMOS circuits. The tool used in this lab is Microwind.

Goals of this lab are:

- Analyze the different delays of CMOS circuits
- Design the layout of given CMOS

Pre-Lab

Delay:

Delay is one of the important factors in CMOS circuits. You will understand the various kinds of delay in this lab.

Some definitions:

- Propagation delay time, tpd = maximum time from the input crossing 50% to the output crossing 50%
- Contamination delay time, tcd = minimum time from the input crossing 50% to the output crossing 50%
- Rise time, tr = time for a waveform to rise from 20% to 80% of its steady-state value
- Fall time, tf = time for a waveform to fall from 80% to 20% of its steady-state value
- Edge rate, trf = (tr + tf)/2

Intuitively, you know that when an input changes, the output will retain its old value for at least the contamination delay and take on its new value in at most the propagation delay.

RC Delay Model:

RC delay models approximate the nonlinear transistor I-V and C-V characteristics with an average resistance and capacitance over the switching range of the gate. The RC delay model treats a transistor as a switch in series with a resistor. A unit nMOS transistor is defined to have effective resistance R. The size of the unit transistor is arbitrary but conventionally refers to a transistor with minimum length and minimum contacted diffusion width (i.e., 4/2 λ). Alternatively, it may refer to the width of the nMOS transistor in a minimum-sized inverter in a standard cell library. An nMOS transistor of k times unit width has resistance R/k because it delivers k times as much current. A unit pMOS transistor has greater resistance, generally in the range of 2R–3R, because of its lower mobility.

Equivalent RC Circuits:

Figure 6.1 shows equivalent RC circuit models for nMOS and pMOS transistors of width k with contacted diffusion on both source and drain. The pMOS transistor has approximately twice the resistance of the nMOS transistor because holes have lower mobility than electrons. The capacitors are shown with VDD as their second terminal because the n-well is usually tied high. However, the behaviour of the capacitor from a delay perspective is independent of the second terminal voltage so long as it is constant.

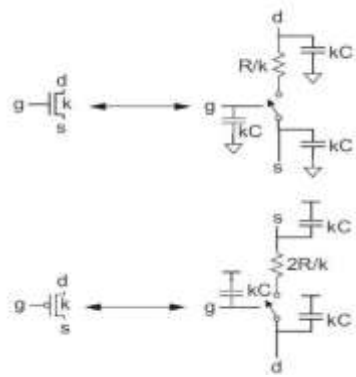


Figure 6.1 Equivalent circuits for transistors

Elmore Delay:

The Elmore delay model estimates the delay from a source switching to one of the leaf nodes changing as the sum over each node i of the capacitance C_i on the node, multiplied by the effective resistance R_{is} on the shared path from the source to the node and the leaf. Estimates Propagation

Delay as the sum over each node in the ladder of the resistance between that node and the source, multiplied by the capacitance on the node:

$$t_{pd} \approx \sum_{\text{nodes } i} R_{i-to-source} C_i$$

$$= R_1 C_1 + \left(R_1 + R_2\right) C_2 + \ldots + \left(R_1 + R_2 + \ldots + R_N\right) C_N$$

RC model shows that delay is a linear function of fanout of a gate. In linear delay model normalized

delay is expressed in terms of two components of delay:

$$d = f + p$$

1. The parasitic delay (p) is the time for a gate to drive its own internal diffusion capacitance (no load)

2. The inverter has three units of diffusion capacitance on the output, so the parasitic delay is $3RC = \tau$

3. The 3-input NAND and NOR each have 9 units of diffusion capacitance on the output, so the parasitic delay is three times as great.

4. The effort delay/stage effort (f) depends on h and g:

$$f = gh$$

Computing Logical Effort:

Logical effort is the ratio of the input capacitance of a gate to the input capacitance of an inverter delivering the same output current. Figure 6.2 shows logical efforts of inverter, NAND and NOR gate.

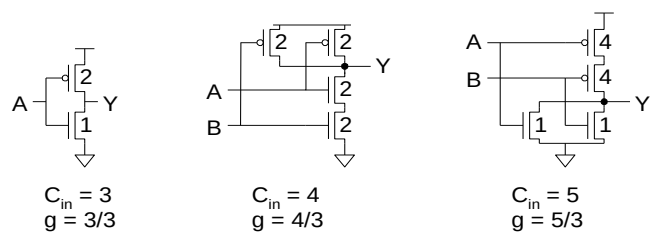


Figure 6.2 Logical efforts

Lab Tasks

Task I: 3-Input NAND gate

a) *Follow* the design of 3-input NAND gate to sketch its schematic diagram with transistor widths chosen to achieve effective rise and fall resistance equal to that of a unit inverter (R).

b) Annotate the gate with its gate and diffusion capacitances. Assume all diffusion nodes are contacted.

c) *Reproduce* 3-input NAND gate for the falling output transition and for the worst-case rising output transition.

d) Estimate $\mathbf{t_{pdf}}$ and $\mathbf{t_{pdr}}$ if the output is loaded with *h* identical NAND gates

e) Estimate contamination delays $\mathbf{t_{pdf}}$ and $\mathbf{t_{pdr}}$ if the output is loaded with *h* identical NAND gates

f) *Follow* the schematic obtained in part c to implement the layout on Microwind

g) *Show* the response of layout using Microwind simulator

h) Compare the delays computed with the delays measured from the layout implemented on Microwind.

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed assigned tasks without any help from the instructor and showed the results appropriately.	4	
Good	The student completed assigned tasks with minimal help from the instructor and showed the results appropriately.	3	
Average	The student could not complete all assigned tasks and showed partial results.	2	
Worst	The student did not complete assigned tasks.	1	

Instructor Signature: _____ Date: _____

LAB # 7:
To implement the layout of combinational dynamic circuits in Microwind and *reproduce* its results

Objective

In this lab students will design and implement the layouts of combinational dynamic circuits. The tool used in this lab is Microwind. The goals for this Lab are:

- Design of combinational dynamic circuits
- Layout design using the tool
- Transistor sizing, logical effort

Pre-Lab

Digital logic is divided into combinational and sequential circuits. Combinational circuits are those whose outputs depend only on the present inputs, while sequential circuits have memory. Generally, the building blocks for combinational circuits are logic gates, while the building blocks for sequential circuits are registers and latches. Static CMOS circuits with complementary nMOS pulldown and pMOS pullup networks are used for the vast majority of logic gates in integrated circuits.

Dynamic Circuits:

Dynamic circuits circumvent the drawbacks of ratioed circuits i-e slow rising transitions, contention on the falling transitions, static power dissipation, and a nonzero VOL by using a clocked pullup transistor rather than a pMOS that is ON. Figure 7.1 compares (a) static CMOS, (b) pseudo-nMOS, and (c) dynamic inverters

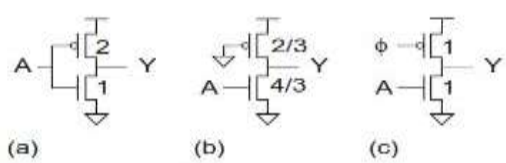
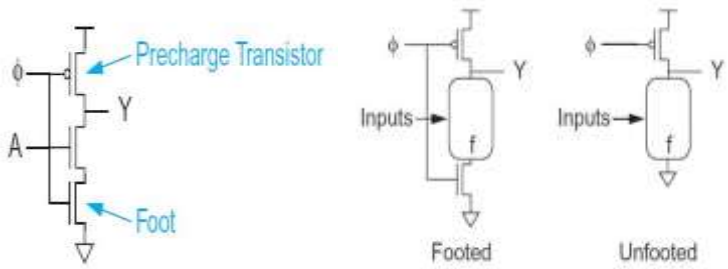


Figure 7.1 Comparison of (a) Static CMOS, (b)Pseudo-nMOS and (c)Dynamic Inverter

Dynamic circuits are the fastest commonly used circuit family because they have lower input capacitance and no contention during switching. They also have zero static power dissipation. However, they require careful clocking, consume significant dynamic power, and are sensitive to noise during evaluation.

In Figure 7.1(c), if the input *A* is 1 during precharge, contention will take place because both the pMOS and nMOS transistors will be ON. When the input cannot be guaranteed to be 0 during precharge, an extra clocked evaluation transistor can be added to the bottom of the nMOS stack to avoid contention as shown in Figure 7.2 (a). The extra transistor is sometimes called a *foot*. Figure 7.3 (b) shows generic *footed* and *unfooted* gates.



(a) Footed Dynamic inverter *(b) Generalized footed and unfooted dynamic gates*

Figure 7.2 dynamic gates

Figure 7.2 estimates the falling logical effort of both footed and unfooted dynamic gates. As usual, the pulldown transistors' widths are chosen to give unit resistance. Precharge occurs while the gate is idle and often may take place more slowly. Therefore, the precharge transistor width is chosen for twice unit resistance. This reduces the capacitive load on the clock and the parasitic capacitance at the expense of greater rising delays. you see that the logical efforts are very low. Footed gates have higher logical effort than their unfooted counterparts but are still an improvement over static logic.

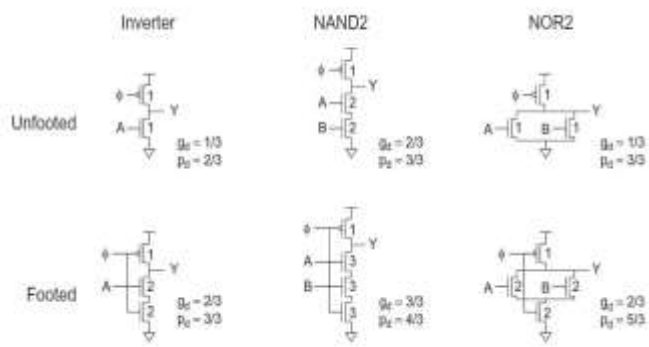


Figure 7.3 Catalog of dynamic gates

Lab Tasks

Task-I: Y= (A. B+C.D)'

(a) *Follow* the function 'Y' to sketch dynamic footed and unfooted circuit. Moreover, label the transistor widths as well.

‘

(b) Compute the logical effort of each gate?

(c) *Follow* the circuits obtained in Part a to implement the layout of dynamic footed and unfooted gates on Microwind.

(d) *Show* the response using Microwind simulator

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____

LAB # 8:
To implement the layout of combinational ratioed logic and *reproduce* the results using Microwind simulator

Objective

In this lab students will design and implement the layouts of combinational ratioed circuits. The tool used in this lab is Microwind. The goals for this Lab are:

- Design of combinational ratioed circuits
- Layout design using the tool
- Transistor sizing, logical effort

Pre-Lab

Ratioed circuits depend on the proper size or resistance of devices for correct operation. In the 1970s and early 1980s before CMOS technologies matured, circuits were often built with only nMOS transistors, as shown in Figure 8.1.

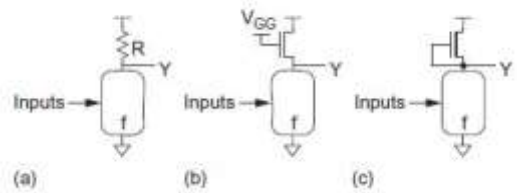


Figure 8.1 nMOS ratioed gates

Ratioed Circuits:

Conceptually, the ratioed gate consists of an nMOS pulldown network and some pullup device called the static load. When the pulldown network is OFF, the static load pulls the output to 1. When the pulldown network turns ON, it fights the static load. The static load must be weak enough that the output pulls down to an acceptable 0. Hence, there is a ratio constraint between the static load and pulldown network. Eventually displaced nMOS logic because the static power became unacceptable as the number of gates increased. However, ratioed circuits are occasionally still useful in special applications.

Pseudo - nMOS Circuits:

Figure 8.2 shows several pseudo-nMOS logic gates. The pulldown network is like that of an ordinary static gate, but the pullup network has been replaced with a single pMOS transistor that is grounded so it is always ON. The pMOS transistor widths are selected to be about 1/4 the strength (i.e., 1/2 the effective width) of the nMOS

pulldown network as a compromise between noise margin and speed; this best size is process-dependent but is usually in the range of 1/3 to 1/6.

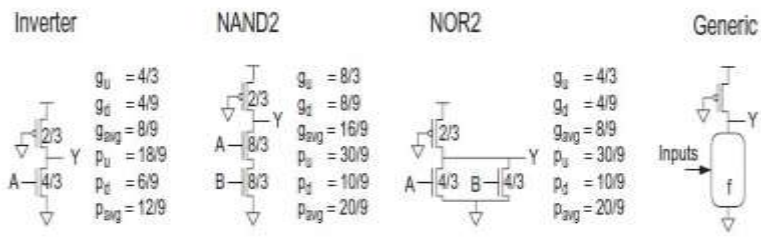


Figure 8.2 Pseudo nMOS logic gates

Calculation of Logical effort of Pseudo - nMOS Circuits:

suppose a complementary CMOS unit inverter delivers current *I* in both rising and falling transitions. For the widths shown, the pMOS transistors produce *I*/3 and the nMOS networks produce 4*I*/3. The logical effort for each transition is computed as the ratio of the input capacitance to that of a complementary CMOS inverter with equal current for that transition. For the falling transition, the pMOS transistor effectively fights the nMOS pulldown. The output current is estimated as the pulldown current minus the pullup current, $(4I/3 – I/3) = I$.

The parasitic delay is also found by counting output capacitance and comparing it to an inverter with equal current. For example, the pseudo-nMOS NOR has 10/3 units of diffusion capacitance as compared to 3 for a unit-sized complementary CMOS inverter, so its parasitic delay pulling down is 10/9. The pullup current is 1/3 as great, so the parasitic delay pulling up is 10/3.

Lab Tasks

Task-I: F = (A(B+C+D) +E.F. G)',

(a) Follow the expression given in function **'F'** to sketch a pseudo nMOS circuit for output **F** and label the transistor widths.

(b) What are the rising, falling, and average logical efforts of each gate?

(c) *Follow* the schematic obtained in part a to implement the layout on Microwind tool

(d) *Show* the response using Microwind simulator

(e) Compare the response of dynamic and ratioed circuit.

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____

LAB # 9:
To implement the layout of D-latch, D- Flip flop and *reproduce* the results using Microwind simulator

Objective

In this lab students will design and implement the layouts of D – Latch and D – Flip flop. The tool used in this lab is Microwind. The goals for this Lab are:

- Design of D – Latch and D – Flip flop
- Layout design using the tool
- Response of each Latch and Flip Flop

Pre-Lab

Conventional CMOS latches are built using pass transistors or tristate buffers to pass the data while the latch is transparent and feedback to hold the data while the latch is opaque.

Conventional CMOS Latches:

Figure 9.1 shows a very simple transparent latch built from a single transistor. It is compact and fast but suffers four limitations. The output does not swing from *rail-to-rail* (i.e., from GND to *VDD*); it never rises above *VDD – Vt*. The output is also *dynamic*; in other words, the output floats when the latch is opaque.

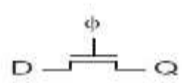


Figure 9.1 simple transparent latch

Figure 9.2 uses a CMOS transmission gate in place of the single nMOS transistor to offer rail to-rail output swings. It requires a complementary clock φ', which can be provided as an additional input or locally generated from φ through an inverter.

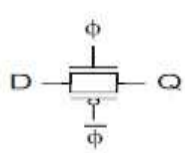


Figure 9.2 Transmission gate-based Latch

A number of alternative latch and flip-flop structures have been used in commercial designs. Different forms of latches are defined in figure 9.3 below.

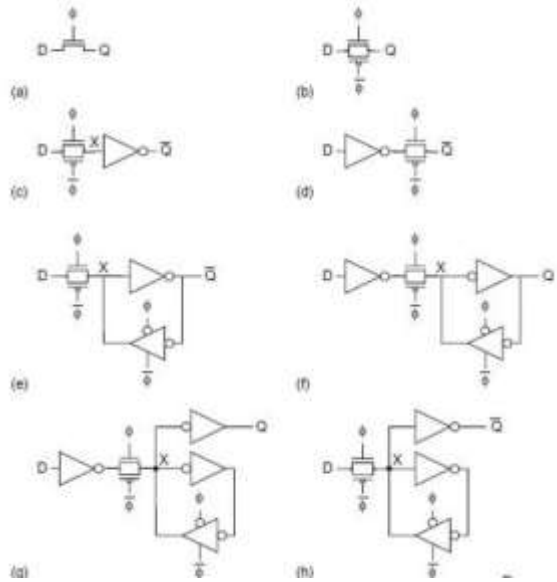


Figure 9.3 Different Transparent Latches

Conventional CMOS Flip-Flops:

You can build Flip flop using latches. Figure 9. 4 (a) shows a dynamic inverting flip-flop built from a pair of back-to-back dynamic latches. Either the first or the last inverter can be removed to reduce delay at the expense of greater noise sensitivity on the unbuffered input or output

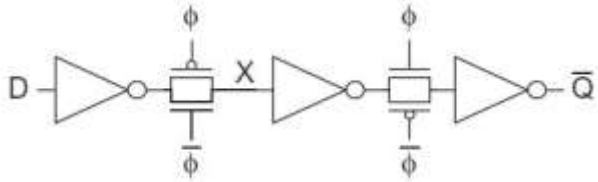


Figure 9.4 (a) Flip Flop

Figure 9. 4 (b) adds feedback and another inverter to produce a noninverting static flip-flop. The PowerPC 603 microprocessor data path used this flip-flop design without the input inverter or Q' output. Most standard cell libraries employ this design because it is simple, robust, compact, and energy efficient.

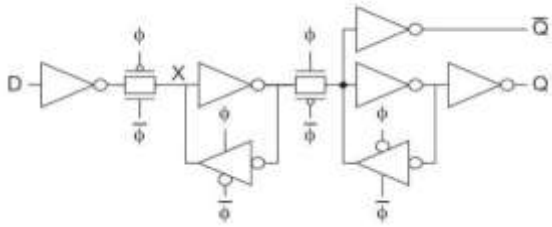


Figure 9.4 (b) Flip Flop

Flip-flops usually take a single clock signal φ and locally generate its complement φ'. If the clock rise/fall time is very slow, it is possible that both the clock and its complement will simultaneously be at intermediate voltages, making both latches transparent and increasing the flip-flop hold time.

Lab Tasks

Task I:

(a) *Follow* the schematic of D- latch to design its layout using

a. NAND
b. NOR
c. Tristate buffer

(b) *Follow* the schematic of Perfect D- latch to design its layout

(c) Calculate / *Show* the propagation delay (t_{pd}), contamination delay (t_{cd}), set up time (t_{setup}), and hold time (t_{hold}).

(d) *Show* the response of each latch using Microwind simulator

Task II:

(a) *Follow* the schematic of D- latch to design its layout using

a. NAND
b. NOR
c. Tristate buffer

(b) Design the layout of rising edge D - flip flop using D- Latch

(c) Calculate / *Show* the propagation delay (t_{pd}), contamination delay (t_{cd}), set up time (t_{setup}), and hold time (t_{hold}).

(d) *Show* the response of flip flop using Microwind simulator

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____

LAB # 10:
To implement the layout of counters and *reproduce* the results using Microwind simulator

Objective

In this lab students will design and implement the layouts of Counter. The tool used in this lab is Microwind. The goals for this Lab are:

- Design of Counters
- Layout design using the tool
- Response of each Counters

Pre-Lab

Counters:

Two commonly used types of counters are binary counters and linear-feedback shift registers. An N-bit binary counter sequences through 2N outputs in binary order. Simple designs have a minimum cycle time that increases with N, but faster designs operate in constant time. An N-bit linear-feedback shift register sequences through up to 2N – 1 outputs in pseudo-random order. It has a short minimum cycle time independent of N, so it is useful for extremely fast counters as well as pseudo-random number generation.

4- Stage Binary Counter:

You will design 4- Stage Binary Counter by using 4 D-registers cascaded. The clocking of each stage is simply carried out by the previous counter, to form an asynchronous counter circuit. The 4-stage binary counter displays number from 0 to 15, using a chain of four D-register cells. The schematic is shown in figure 10.1

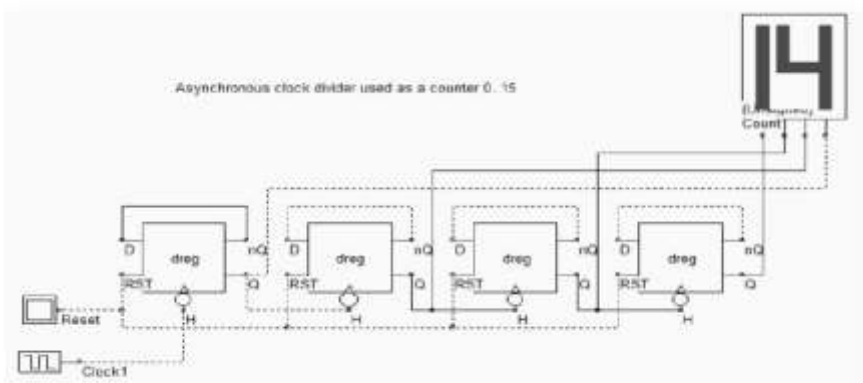


Figure 10.1 Schematic of a 4-stage binary counter

Ring Counter:

A ring counter consists of an M-bit shift register with the output fed back to the input, as shown in Figure 10.2. On reset, the first bit is initialized to 1 and the others are initialized to 0. TC toggles once every M cycles. Ring counters are a convenient way to build extremely fast pre-scalars because there is no logic between flip-flops, but they become costly for larger M.

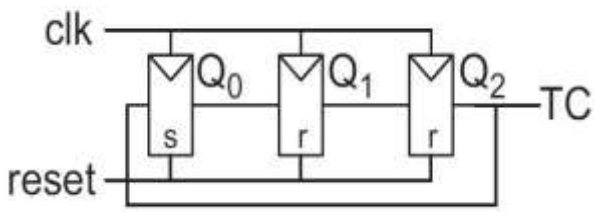


Figure 10.2 Ring Counter

A Johnson or Mobius counter is similar to a ring counter but inverts the output before it is fed back to the input, as shown in Figure 10.3. The flip-flops are reset to all zeros and count through 2M states before repeating.

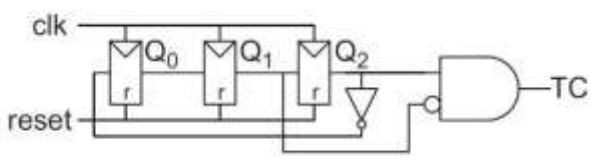


Figure 10.3 Johnson Counter

Lab Tasks

Task I:

Follow the steps to design the Layout of 4- Stage Binary Counter

a) Design the layout of Not gate

b) Design the layout of D – Latch (Use Lab # 9 D – Latch)

c) Calculate the number of transistors used in D – Latch

d) Design the layout of D – register/Flip flop

e) Design the layout of 4 D – register by following the schematic given in figure 10.1.

f) *Show* the response of 4- Stage Binary Counter using Microiwnd simulator.

Task II:

2. *Follow* the steps to design the Layout of 4 - bit ring counter using D- Flip flop as mentioned in figure 10.2

a) *Show* the response of 4 - bit ring counter using simulator.

Task III:

3. *Follow* the steps to design the Layout of 4 - bit Johnson counter using D- Flip flop as mentioned in figure 10.3

a) *Show* the response of 4 - bit Johnson counter using simulator.

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

4. **Instructor Signature:_____ Date: _____**

LAB # 11:
To implement the layout of registers and *reproduce* its results using Microwind simulator

Objectives

In this lab students will design and implement the layouts of register. The tool used in this lab is Microwind. The goals for this Lab are:

- Design of Register
- Layout design using the tool
- Response of each Counters

Pre-Lab

Shift registers are a type of sequential logic circuit, mainly for storage of digital data. They are a group of flip-flops connected in a chain so that the output from one flip-flop becomes the input of the next flip-flop. Most of the registers possess no characteristic internal sequence of states. All flip-flop is driven by a common clock, and all are set or reset simultaneously. A shift register is commonly used in signal-processing applications to store and delay data.

The storage capacity of a register is the total number of bits (1 or 0) of digital data it can retain. Each stage (flip flop) in a shift register represents one bit of storage capacity. Therefore, the number of stages in a register determines its storage capacity as shown in figure 11.1.

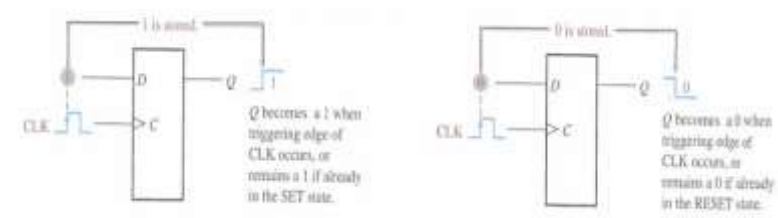


Figure 11.1 The flip flop as a storage element

Types of shift registers:

1. Serial In - Serial Out Shift Registers
2. Serial In - Parallel Out Shift Registers
3. Parallel In - Serial Out Shift Registers
4. Parallel In - Parallel Out Shift Registers

1. Serial In - Serial Out Shift Registers

The serial in/serial out shift register accepts data serially i-e , one bit at a time on a single line. It produces the stored information on its output also in serial form. 4 -bit serial shift register is shown in figure 11.2.

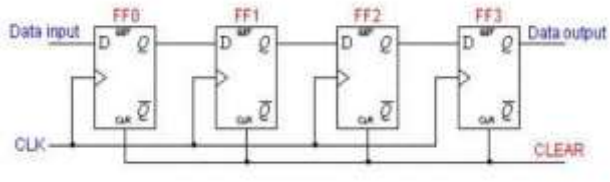


Figure 11.2 4 -bit SISO shift register

2. Serial In - Parallel Out Shift Registers:

For this kind of register, data bits are entered serially in the same manner as discussed in the last section. The difference is the way in which the data bits are taken out of the register. Once the data are stored, each bit appears on its respective output line, and all bits are available simultaneously. A construction of a four-bit serial in - parallel out register is shown below in figure 11.3

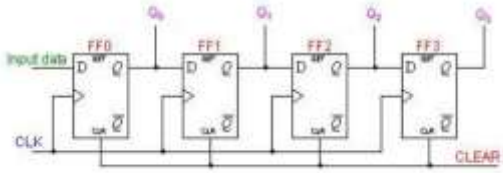


Figure 11.3 4 -bit SIPO shift register

3. Parallel In – Serial Out Shift Registers:

The parallel-in/ serial-out shift register stores data, shifts it on a clock by clock basis, and delays it by the number of stages times the clock period. In addition, parallel-in/ serial-out really means that we can load data in parallel into all stages before any shifting ever begins. This is a way to convert data from a parallel format to a serial format.

4. Parallel In – Parallel Out Shift Registers:

For parallel in - parallel out shift registers, all data bits appear on the parallel outputs immediately following the simultaneous entry of the data bits. The following circuit in figure 11.5 is a four-bit parallel in - parallel out shift register constructed by D flip-flops.

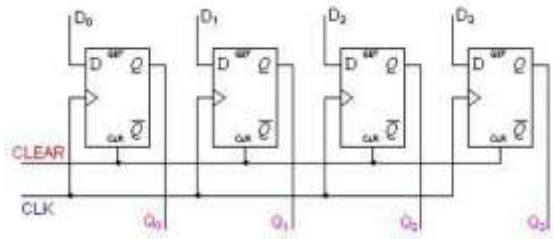


Figure 11.5 4 -bit PIPO shift register

Lab Tasks

Task I:

Follow the steps given below to design the Layout of 4- bit Serial In - Parallel Out Shift Registers as shown in Figure 11.3

a) Design the layout of rising edge D – Flip Flop (use FF of lab # 9)

b) Calculate the number of transistors used in D – Flip Flop

c) Design the layout of 4- bit Serial In - Parallel Out Shift Registers by following the block diagram given in figure 11.3

d) *Show* the response of 4- bit Serial In - Parallel Out Shift Registers.

Task II:

a) *Follow* the block diagram of universal shift register shown in Figure 11.5.5 to design its layout.

b) Design Mux using various methods.

c) Design D-FF using various methods.

d) *Show* the perfect method to design the layout of universal shift register

e) *Show* the output using Microwind simulator

Mode Control

s_1	s_0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

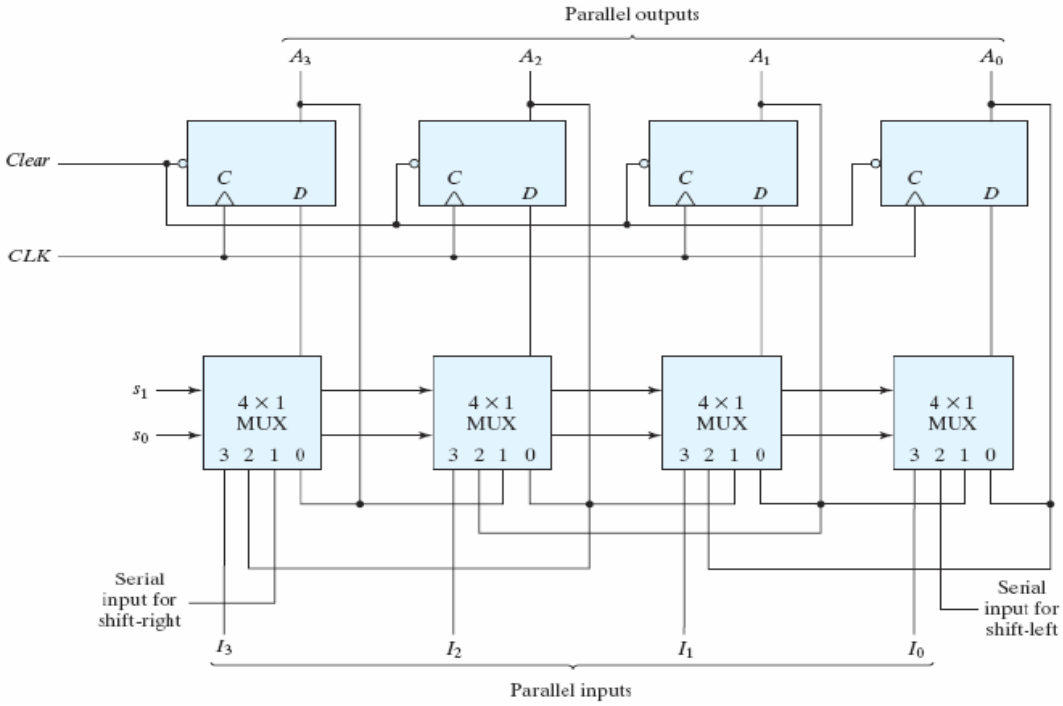


Figure 11.5.5: Universal Shift Register

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Rubric for Lab Assessment

Instructor Signature: _____ Date: _____

LAB # 12: [Open Ended Lab]
To implement the layout of data path components and *reproduce* the results using Microwind simulator

Objectives

In this lab students will design and implement the layouts of different data path components. The tool used in this lab is Microwind. The goals for this Lab are:

- Design of data path
- Layout design using the tool
- Response of each data path components

Pre-Lab

CMOS system design consists of partitioning the system into subsystems having several types like data path operator, memory elements, control structures and special-purpose cells. Many options exist that make trade-offs between speed, density, programmability, ease of design, and other variables. Datapath operators benefit from the structured design principles of hierarchy, regularity, modularity, and locality. They may use *N* identical circuits to process *N*-bit data. Related data operators are placed physically adjacent to each other to reduce wire length and delay. Generally, data is arranged to flow in one direction, while control signals are introduced in a direction orthogonal to the dataflow. Common data path operators considered in this lab include adders, subtractors and comparators.

Adder:

A Full Adder is an important building block of arithmetic circuits in a system. Full adder accepts three inputs and produces the outputs sum and carry by adding the binary bits with the help of logic gates. Equations of sum and carry are listed below. The carry gate is also called a majority gate because it produces a 1 if at least two of the three inputs are 1.

$$\boldsymbol{S = AB'C' + A'BC' + A'B'C + ABC} \qquad (\mathbf{1})$$

$$\boldsymbol{= (A \oplus B) \oplus C = P \oplus C}$$

$$\boldsymbol{Cout = AB + AC + BC} \qquad (\mathbf{2})$$

$$\boldsymbol{= AB + C\,(A + B)}$$

$$\boldsymbol{= \left(A'B' + C'(A' + B')\right)'}$$

$$\boldsymbol{= MAJ\,(A, B, C)} \qquad (\mathbf{3})$$

Lab Tasks

Task I: Full Adder Design

a) *Follow* equations (1) and (2) to design the CMOS full adder at the transistor level

b) Calculate the number of transistors used in designing the full adder

c) *Follow* the design of full adder using transmission gates to design its layout

d) Calculate the number of transistors used in designing the full adder using transmission gates

e) *Follow* the design of full adder using mirror logic (Mirror adder) to design its layout

f) Calculate the number of transistors used in designing the mirror full adder

g) *Follow* the steps to implement the layout of full adder in Microwind using each technique mentioned above

h) *Show* the response of each technique by using simulator

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____